A Comparative Study of Recurrent and Self-Attentive Architectures for News Topic Classification

Sequential and Transformer models implemented from first principles and evaluated on the AG News corpus

---

Case Study Report

Transformer Modeling and Sequence Learning

Topic 2 — News Topic Classification Using Sequential and Transformer Models

---

Research question	How do recurrent networks and Transformers compare at classifying short news texts?
Dataset	AG News — 120,000 training / 7,600 test articles four classes: World, Sports, Business, Sci/Tech
Models	Bidirectional LSTM (hand-written cell and recurrence) Transformer encoder (hand-written self-attention)
Headline result	91.14% vs 91.05% test accuracy; difference not statistically significant ($p = 0.754$)

Source code and executed notebook:

`https://github.com/Ashwin-3cS/transformer-t1`

October 30, 2026

Abstract

This report presents a controlled empirical comparison of two families of sequence model on the task of four-class news topic classification. A bidirectional Long Short-Term Memory network and a Transformer encoder are implemented from first principles — the LSTM gating equations, the unrolled recurrence, scaled dot-product attention, multi-head projection, sinusoidal positional encoding and layer normalisation are all written by hand rather than invoked from a deep learning library — and are trained and evaluated under an experimental protocol designed so that architecture is the only free variable.

The study uses the AG News corpus. Following the case-study brief, a stratified subset of 60,000 records is drawn from the 120,000-document training file and split into 54,000 training and 6,000 validation examples, while the full official 7,600-document test set is held back for a single final evaluation. Both models consume an identical vocabulary, identical tensors and an identical training budget, and are matched to within 0.19% on non-embedding parameter count.

Before any training is performed, nine numerical checks verify the hand-written layers against reference implementations and against invariants that a correct implementation must satisfy. All nine pass. This verification step is reported as a substantive part of the methodology rather than as an implementation detail, because the characteristic failure mode of a hand-written layer is not a crash but a silently degraded result.

The principal finding is negative and is reported as such. The two architectures achieve 91.14% and 91.05% test accuracy respectively, a difference of 0.09 percentage points. McNemar's exact test on the paired predictions yields $p = 0.754$, and the models agree on 94.78% of test documents, so the difference cannot be distinguished from sampling noise. The two models further exhibit an identical per-class difficulty ordering and share the same dominant error, confusing Business with Sci/Tech in approximately 9.4% of Business documents. The report argues that this equivalence is a consequence of document length: AG News items have a median of 39 tokens, and self-attention's structural advantage — constant path length between distant positions — has little to act upon at that scale.

A secondary finding concerns computational cost and runs contrary to the usual expectation. On the CPU-only hardware used here, and at a sequence length of 64 tokens, the recurrent model trained 1.67 times faster per epoch than the Transformer. The report analyses why, distinguishing the asymptotic argument for attention — which concerns parallelisability and growth in sequence length — from the constant factors that in fact determine wall-clock time in this regime.

Keywords: text classification, recurrent neural networks, long short-term memory, self-attention, Transformer, AG News, McNemar's test, reproducible machine learning.

Contents

Abstract **ii**

1 Introduction **1**

1.1 Background and motivation 1
1.2 Problem statement 1
1.3 Research question 2
1.4 Objectives 2
1.5 Scope and delimitations 3
1.6 Contributions 3
1.7 Organisation of this report 4

2 Literature Review **5**

2.1 Overview 5
2.2 The sparse feature era 5
2.3 Distributed word representations 5
2.4 Recurrent neural networks 6
2.5 Long Short-Term Memory 6
2.6 Bidirectionality 6
2.7 Attention 7
2.8 The Transformer 7
2.9 Pretrained language models 7
2.10 Large language models and the current landscape 8
2.11 Prior results on AG News 8
2.12 Research gap 9

3 Theoretical Background **10**

3.1 Problem formalisation 10
3.2 Embedding 10
3.3 Recurrent networks and the vanishing gradient problem 11
3.3.1 The simple recurrent formulation 11

3.3.2 Backpropagation through time . . . 11
3.3.3 The pathology . . . 11
3.4 Long Short-Term Memory . . . 12
3.4.1 Gating equations . . . 12
3.4.2 Why this solves the gradient problem . . . 12
3.4.3 Fused gate computation . . . 12
3.4.4 Parameter count . . . 13
3.5 Bidirectional recurrence . . . 13
3.6 Pooling . . . 13
3.7 Attention . . . 13
3.7.1 Scaled dot-product attention . . . 13
3.7.2 Why the $\sqrt{d_k}$ scaling is necessary . . . 14
3.7.3 Multi-head attention . . . 14
3.7.4 Masking . . . 14
3.8 Positional encoding . . . 15
3.9 Layer normalisation . . . 15
3.10 Residual connections and block ordering . . . 15
3.11 Complexity analysis . . . 16

4 Dataset and Methodology **17**
4.1 The AG News corpus . . . 17
4.2 Data cleaning . . . 17
4.3 Experimental design . . . 18
4.3.1 The controlled subset . . . 18
4.3.2 The four experimental controls . . . 18
4.3.3 The one quantity that is not shared . . . 19
4.4 Preprocessing . . . 19
4.4.1 Tokenisation . . . 19
4.4.2 Vocabulary . . . 19
4.4.3 Sequence length . . . 19
4.4.4 Encoding . . . 20
4.5 Evaluation metrics . . . 21
4.5.1 Accuracy and F_1 . . . 21
4.5.2 Confusion matrix . . . 21
4.5.3 McNemar's test . . . 21
4.6 Experimental environment . . . 22

5 Implementation **23**

5.1 The from-scratch constraint . . . 23
5.2 Model A: the bidirectional LSTM . . . 24
5.2.1 The cell . . . 24
5.2.2 The recurrence, and the masking problem . . . 24
5.2.3 The classifier . . . 25
5.3 Model B: the Transformer encoder . . . 26
5.3.1 Positional encoding and layer normalisation . . . 26
5.3.2 Multi-head self-attention . . . 26
5.3.3 Encoder block and classifier . . . 27
5.4 Training harness . . . 27
5.5 Architecture configuration and parameter budget . . . 28

6 Verification of the Hand-Written Layers 29
6.1 Why verification is methodologically necessary . . . 29
6.2 The reference implementations . . . 29
6.3 Checks 1–2: equivalence against reference implementations . . . 30
6.3.1 Check 1 — the LSTM cell . . . 30
6.3.2 Check 2 — layer normalisation . . . 30
6.4 Check 3: attention against an independent formulation . . . 30
6.5 Checks 4–5: mathematical invariants . . . 31
6.6 Checks 6–7: padding invariance . . . 31
6.7 Checks 8–9: gradient viability . . . 32
6.8 Results . . . 32
6.9 What this suite does not establish . . . 32

7 Results 33
7.1 Training dynamics . . . 33
7.2 Headline results . . . 33
7.3 Per-class performance . . . 34
7.4 Confusion structure . . . 35
7.5 Statistical significance . . . 35
7.6 Computational cost . . . 36
7.7 Qualitative error analysis . . . 37
7.7.1 Documents both models misclassify . . . 37
7.7.2 Documents on which the models disagree . . . 37

8 Discussion 40
8.1 Answering the research question . . . 40
8.1.1 They are indistinguishable in accuracy . . . 40

8.1.2 The explanation is document length . . . 40
8.1.3 They fail in the same way . . . 41
8.1.4 Cost is where they differ, and the finding inverts expectation . . . 41
8.2 Comparison with published results . . . 41
8.3 Implications . . . 42
8.4 Threats to validity . . . 42
8.4.1 Internal validity . . . 42
8.4.2 External validity . . . 42
8.4.3 Construct validity . . . 42

9 Conclusion and Future Work **44**
9.1 Summary . . . 44
9.2 Findings . . . 44
9.3 Contributions revisited . . . 44
9.4 Future work . . . 45
9.5 Closing remarks . . . 45

References **47**

A Reproduction **48**
A.1 Environment . . . 48
A.2 Data access . . . 48
A.3 Execution . . . 48
A.4 Outputs . . . 49
A.5 Determinism . . . 49

B Complete Classification Reports **50**
B.1 Model A — bidirectional LSTM . . . 50
B.2 Model B — Transformer encoder . . . 50
B.3 Confusion matrices, absolute counts . . . 50

C Source Listings **52**
C.1 Transformer encoder, complete . . . 52
C.2 Bidirectional LSTM classifier, complete . . . 54

D Verification Suite Output **55**

E Summary of Key Figures **56**

List of Figures

4.1 Distribution of document lengths 20

7.1 Learning curves 34
7.2 Per-class F1 comparison 35
7.3 Confusion matrices 36

List of Tables

2.1 Representative published results on AG News 8

3.1 Complexity comparison of recurrent and self-attentive layers 16

4.1 Composition of the AG News corpus as distributed. 17
4.2 Data splits used in the experiment. The test set is the full official file and is used once, for the final evaluation reported in Chapter 7. 18
4.3 Token-length statistics of the training split. 20
4.4 Contingency structure for McNemar's test. 21
4.5 Hardware and software environment. All timing results in Chapter 7 are specific to this configuration. 22

5.1 Components implemented from first principles, with the equations they realise. 23
5.2 Training configuration. All values except the learning rate are shared. 27
5.3 Architectural hyperparameters. 28
5.4 Parameter budget 28

6.1 Verification results 32

7.1 Training summary. Both models stopped early, well inside the epoch cap of eight. 33
7.2 Headline test-set results 34
7.3 Per-class results 34
7.4 Largest off-diagonal confusions, as a proportion of the true class. 35
7.5 McNemar contingency table 36
7.6 Computational cost 37
7.7 Documents misclassified by both models 38
7.8 Documents on which the models disagree 39

B.1 Confusion matrix, bidirectional LSTM. Rows are true classes. 51
B.2 Confusion matrix, Transformer encoder. Rows are true classes. 51

E.1 All headline quantities reported in this study. 56
E.2 Shared experimental quantities. 57

Chapter 1

Introduction

1.1 Background and motivation

Automatic topic classification of news text is one of the oldest and most durable applications of natural language processing. News agencies route incoming wire copy to desks; aggregators assemble sections; recommendation systems filter by interest; archives index by subject. In each case the underlying computational problem is the same: given a short passage of text, assign it to one of a small set of editorial categories.

What has changed repeatedly over the past three decades is not the problem but the representation. Early systems treated a document as an unordered bag of words and applied a linear classifier to sparse count features. The introduction of distributed word representations replaced sparse indicators with dense vectors that encoded distributional similarity. Recurrent neural networks then made it possible to model a document as a *sequence* rather than a set, processing tokens in order and maintaining a hidden state that in principle summarises everything seen so far. The Transformer subsequently discarded recurrence altogether, replacing the sequential state with an attention mechanism that relates every position to every other position directly.

The Transformer's dominance in contemporary NLP is not in dispute. What is less often examined carefully is *where* its advantage actually comes from, and whether that advantage is present in every task to which the architecture is applied. The mechanism that makes self-attention powerful — a constant-length computational path between any two positions in a sequence — is explicitly a statement about *long-range* dependency. It is reasonable to ask what remains of that advantage when the sequences in question are not long.

This question is not merely academic. A great many deployed text classification systems operate on short inputs: search queries, product titles, support ticket subjects, headlines. If the architectural advantage of self-attention is contingent on sequence length, then the engineering decision for such systems cannot be read off from benchmark leaderboards built on longer documents.

1.2 Problem statement

The task addressed in this report is four-class topic classification of short news documents. Given a news item consisting of a headline and a one-sentence description, the system must

assign exactly one of four labels: *World*, *Sports*, *Business*, or *Science/Technology*.

Two properties of this task make it a suitable vehicle for the architectural comparison at hand.

First, the documents are short. In the corpus used here the median document is 39 tokens long. This places the task squarely in the regime where the long-range-dependency argument for self-attention is weakest, and therefore where a direct comparison is most informative.

Second, the label set has four classes rather than two. A binary sentiment task yields a single accuracy number and a largely uninformative confusion structure. Four classes produce a confusion matrix with twelve off-diagonal cells, which permits a far more detailed account of *how* a model fails rather than merely how often. As will be shown in Chapter 7, the error structure turns out to be considerably more informative than the headline accuracy.

1.3 Research question

The report addresses a single primary research question:

> **How do recurrent networks and Transformers compare at classifying short news texts?**

This decomposes into four subsidiary questions, each of which is answered empirically:

1. **Accuracy.** Does either architecture achieve higher classification accuracy, and is any observed difference statistically distinguishable from sampling noise?
2. **Error structure.** Do the two architectures fail on the same documents and the same class boundaries, or do they exhibit different inductive biases?
3. **Cost.** How do the architectures compare in training time, inference latency and parameter efficiency at matched capacity?
4. **Explanation.** Whatever differences or equivalences are observed, what property of the task accounts for them?

1.4 Objectives

The study was conducted with the following concrete objectives.

1. To implement a bidirectional LSTM classifier from first principles, writing the gating equations, the cell-state recurrence and the bidirectional unrolling explicitly rather than invoking a library primitive.
2. To implement a Transformer encoder classifier from first principles, writing scaled dot-product attention, multi-head projection and merging, sinusoidal positional encoding, layer normalisation and the residual block structure explicitly.
3. To verify both implementations numerically against reference implementations and against invariants that any correct implementation must satisfy, *before* drawing any experimental conclusion from them.

4. To design a training and evaluation protocol in which architecture is the only free variable, holding data, preprocessing, capacity and training budget fixed.

5. To evaluate both models on a held-out test set using metrics appropriate to a balanced multi-class problem, including a test of statistical significance on the paired predictions.

6. To analyse the computational cost of both architectures and to interpret the result in terms of the structural properties of recurrence and attention.

1.5 Scope and delimitations

It is as important to state what this study does *not* attempt as what it does.

Training from scratch, not transfer learning. Both models are trained from randomly initialised embeddings on the task data alone. No pretrained representations are used. This is a deliberate scoping decision: it isolates the architectural question. A comparison involving a pretrained Transformer would primarily measure the value of pretraining, which is a different and already well-established result.

Matched capacity, not maximum performance. The models are deliberately sized to have near-identical non-embedding parameter counts. Neither is tuned to its maximum achievable accuracy on AG News. The objective is a fair comparison, not a leaderboard entry.

A controlled subset, not the full corpus. Following the case-study brief, 60,000 of the 120,000 available training records are used. The consequences of this choice are discussed in Section 8.4.

One corpus, one language, one document length regime. All conclusions are drawn from English-language short news text. The report is explicit, particularly in Chapter 8, about which findings are expected to generalise and which are specific to this regime.

1.6 Contributions

This report makes the following contributions.

1. **A fully transparent implementation** of both architectures, in which every sequence-modelling operation is written explicitly and is available for inspection. Nothing of consequence is hidden inside a library call.

2. **A verification methodology for hand-written layers.** Nine numerical checks are specified, motivated and executed, covering equivalence against reference implementations, mathematical invariants, masking correctness and gradient viability. The report argues that such verification is not optional for from-scratch work, because the characteristic failure mode is silent degradation rather than an observable error.

3. **A genuinely controlled comparison**, in which data, preprocessing, capacity and training budget are held fixed and only the architecture varies.

4. **A statistically disciplined negative result.** The observed accuracy difference is 0.09 percentage points. Rather than reporting a winner, the study applies McNemar's exact

test, finds $p = 0.754$, and concludes that the architectures are indistinguishable on this task. The report treats the significance test as essential rather than decorative.

5. **A cost finding that contradicts the expected result**, together with an analysis that distinguishes the asymptotic argument for attention from the constant factors that determine wall-clock time at short sequence lengths.

1.7 Organisation of this report

The remainder of the report is structured as follows.

Chapter 2 reviews the development of text classification from sparse feature methods through recurrent architectures to self-attention, and surveys published results on the AG News benchmark in order to situate the present work.

Chapter 3 develops the mathematical background: the recurrent formulation and its gradient pathology, the LSTM gating solution, the attention mechanism and its scaling, and a complexity analysis that formalises the structural trade-off between the two architectures.

Chapter 4 describes the corpus, the data cleaning required, the controlled experimental design, the preprocessing pipeline and the evaluation metrics.

Chapter 5 presents the implementation of both models, with attention to the specific points at which a from-scratch implementation is most likely to go wrong.

Chapter 6 presents the nine verification checks, their motivation and their results.

Chapter 7 reports the experimental results: training dynamics, accuracy, per-class performance, confusion structure, statistical significance, computational cost and a qualitative error analysis.

Chapter 8 interprets the results, answers the research question, compares the findings with published work, and states the threats to validity.

Chapter 9 concludes and identifies future work.

The appendices contain the full source listings of the hand-written components, the complete classification reports, and instructions for reproducing every number in this document.

Chapter 2

Literature Review

2.1 Overview

This chapter traces the development of neural text classification in order to establish where the present comparison sits. The narrative is organised around a single recurring question: *how is word order represented?* Each major advance in the field can be read as a different answer to that question, and the two architectures compared in this report represent the two most recent answers.

2.2 The sparse feature era

The earliest statistical approaches to text classification represented a document as a vector of term counts or weighted term frequencies, discarding word order entirely. The bag-of-words model, typically weighted by term frequency–inverse document frequency (TF–IDF), paired with a linear classifier such as multinomial Naive Bayes or a support vector machine, formed the standard baseline for many years [8].

These methods remain surprisingly competitive on topic classification specifically. Topic is largely signalled by lexical choice: the presence of the words *quarterly*, *earnings* and *shares* is close to sufficient evidence for a business story, irrespective of the order in which those words appear. This observation matters for the present study, because it implies that topic classification is a task where sequential modelling has comparatively little to contribute — a point returned to in Chapter 8.

The principal weaknesses of the sparse approach are the dimensionality of the feature space, the absence of any notion of semantic similarity between distinct terms, and the inability to represent negation, modification or any other order-dependent phenomenon.

2.3 Distributed word representations

The second weakness was addressed by distributed representations. Word2Vec [16] and GloVe [19] learn dense, low-dimensional vectors such that distributionally similar words occupy nearby regions of the space. A document can then be represented by, for example, the mean of its word vectors — a representation that generalises across synonyms in a way that sparse indicators cannot.

Word order, however, remains unmodelled. Averaging word vectors is still a bag-of-words operation. The FastText classifier [9] demonstrated that this simple approach, with the addition of character n-gram features, achieves strong results on topic classification at a tiny fraction of the computational cost of neural sequence models — again suggesting that the marginal value of sequence modelling on this class of task is modest.

2.4 Recurrent neural networks

Recurrent neural networks introduced genuine sequential modelling. An RNN maintains a hidden state h_t that is updated at each timestep as a function of the current input and the previous state, so that in principle h_T summarises the entire sequence.

In practice, the simple recurrent formulation suffers from the vanishing and exploding gradient problem, analysed by Bengio et al. [3] and later in detail by Pascanu et al. [17]. Because backpropagation through time multiplies Jacobian matrices once per timestep, gradient magnitude decays or grows geometrically with the distance over which credit must be assigned. The practical consequence is that simple RNNs fail to learn dependencies spanning more than a handful of timesteps. The formal statement of this result is developed in Section 3.3.

2.5 Long Short-Term Memory

The Long Short-Term Memory architecture of Hochreiter and Schmidhuber [7] addresses the gradient pathology directly. LSTM introduces a separate *cell state* that is modified by additive rather than multiplicative interactions, and three multiplicative gates — input, forget and output — that control the flow of information into, within and out of the cell. Gers et al. [6] subsequently added the forget gate in the form now considered standard.

The critical structural property is that the cell state path from c_{t-1} to c_t passes through an addition and an elementwise multiplication by the forget gate, rather than through a weight matrix and a saturating nonlinearity. When the forget gate is near one, gradients propagate backwards along this path essentially unattenuated. This is the mechanism by which LSTM extends the effective credit-assignment horizon from a handful of timesteps to hundreds.

The Gated Recurrent Unit [4] simplifies the LSTM to two gates and a single state vector, with broadly comparable empirical performance and fewer parameters. The present study uses LSTM rather than GRU because its gate structure is the canonical formulation and its correspondence with the reference implementation used for verification (Chapter 6) is exact.

2.6 Bidirectionality

A unidirectional recurrent network computes a representation of each position conditioned only on the preceding context. For classification — where the entire document is available at inference time and there is no requirement to generate left to right — this is an unnecessary restriction. Bidirectional recurrent networks [20] run two independent recurrences, one in each direction, and concatenate the resulting representations, so that each position is contextualised by both its left and right neighbourhoods.

Bidirectionality is standard in recurrent text classification and is used in the present study. Note that it doubles the sequential computation, which is relevant to the cost analysis in

Chapter 7.

2.7 Attention

Attention was introduced by Bahdanau et al. [2] in the context of neural machine translation, as a mechanism to relieve the bottleneck of compressing an entire source sentence into a single fixed-length vector. Rather than conditioning on a single summary state, the decoder computes a weighted combination of all encoder states, with weights determined by a learned compatibility function between the decoder's current state and each encoder position.

Luong et al. [13] simplified the compatibility function to a dot product and systematised the variants. The essential insight — that a model can learn *where to look* rather than being forced to summarise everything into one vector — proved far more general than its original application.

2.8 The Transformer

Vaswani et al. [22] took the decisive step of removing recurrence entirely. The Transformer computes representations using *self*-attention, in which every position attends to every other position in the same sequence, combined with position-wise feed-forward layers, residual connections and layer normalisation [1].

Two properties of this design are central to the argument of this report.

Constant path length. In a recurrent network, information must traverse $O(n)$ sequential steps to travel between positions n apart. In self-attention, every pair of positions is connected directly, so the path length is $O(1)$. This is the structural basis of the Transformer's advantage on long-range dependency.

Parallelism. Recurrence is inherently sequential: step t cannot be computed until step $t-1$ is complete. Self-attention has no such constraint; all positions are computed simultaneously as a batched matrix product. On parallel hardware this translates directly into reduced wall-clock time per training step.

Because self-attention is permutation-invariant, positional information must be injected explicitly. The original work uses fixed sinusoidal encodings; learned positional embeddings are a common alternative. The pre-norm variant of the residual block, in which layer normalisation is applied before rather than after each sublayer, was shown by Xiong et al. [23] to yield better-conditioned gradients at initialisation and to remove the need for a learning-rate warmup schedule. The present study uses the pre-norm arrangement for precisely this reason.

2.9 Pretrained language models

BERT [5] demonstrated that a Transformer encoder pretrained on a large unlabelled corpus with a masked language modelling objective, then fine-tuned on a downstream task, substantially outperforms task-specific architectures trained from scratch. The paradigm now dominates the field, with RoBERTa [11] and numerous successors refining the recipe.

It is important to be precise about what this result establishes. The gain attributable to BERT on a task such as AG News comes overwhelmingly from *pretraining* — from represen-

tations learned on billions of words of unlabelled text — rather than from self-attention as an architectural primitive. Conflating the two is a common error, and the present study is explicitly designed to avoid it: by training both architectures from scratch on identical data, the comparison isolates the architectural contribution.

2.10 Large language models and the current landscape

The most recent development is the application of large generative language models to classification, either zero-shot, few-shot, or via instruction tuning. A 2024 study on text classification in the age of large language models [15] — identified in the case-study brief as the related work for this topic — examines this landscape and reports a finding directly relevant here: for well-specified classification tasks with adequate labelled data, compact supervised models remain competitive with, and far cheaper than, large generative models. The persistent value of small task-specific classifiers is a recurring conclusion in this literature, and it motivates continued careful study of exactly the architectures compared in this report.

2.11 Prior results on AG News

AG News was introduced as a classification benchmark by Zhang et al. [24], who used it to evaluate character-level convolutional networks. It has since become a standard four-class topic classification benchmark. Table 2.1 summarises representative published accuracies.

Table 2.1: Representative published test accuracies on the AG News benchmark. Figures are as reported in the cited work and are not strictly comparable, since training budgets, preprocessing and the use of pretraining differ. They are presented to situate the present study rather than to rank methods.

Approach	**Family**	**Reported accuracy**
Bag of words + linear classifier [24]	Sparse features	$\approx 88.8\%$
FastText [9]	Embedding average	$\approx 92.5\%$
Character-level CNN [24]	Convolutional	$\approx 90.5\%$
Word-level CNN / LSTM baselines [24]	Neural sequence	≈ 90–92%
BERT-base fine-tuned [5, 11]	Pretrained Transformer	≈ 94–95%

Three observations follow from this table and are important context for the results reported later.

First, the spread between a simple bag-of-words classifier and a fine-tuned BERT model is roughly six percentage points. This is a comparatively narrow band, and it indicates that AG News is a task on which lexical evidence carries most of the signal.

Second, the models in the middle of this range — FastText, character CNNs, word-level recurrent and convolutional baselines — are clustered tightly around 90–92%. Architectures of very different character arrive at very similar accuracy, which is itself suggestive.

Third, the models at the top of the range are distinguished by pretraining, not by architecture. The present study, which trains from scratch, should therefore be expected to land in the 90–92% band, and does.

2.12 Research gap

The literature establishes that Transformers outperform recurrent networks on a wide range of tasks, and that pretrained Transformers outperform everything. What is less well documented is a careful, controlled comparison of the two architectures *at matched capacity*, *without pretraining*, on *short* text, with *statistical testing* of the resulting difference.

Published comparisons frequently differ in parameter count, training budget, preprocessing or tokenisation between the systems being compared, which makes the attribution of any observed difference to architecture unsafe. Significance testing of classification differences is uncommon, despite accuracy differences of under one percentage point being routinely reported as improvements.

This report addresses that gap for one specific and practically important regime: short-text topic classification. It holds data, preprocessing, capacity and training budget fixed; it verifies both implementations numerically before drawing conclusions from them; and it applies a paired significance test to the resulting predictions.

Chapter 3

Theoretical Background

This chapter develops the mathematics of the two architectures. The treatment is deliberately explicit, because the implementations in Chapter 5 follow these equations term by term, and the verification in Chapter 6 checks them numerically.

3.1 Problem formalisation

Let $\mathcal{V}$ be a finite vocabulary and let a document be a sequence of tokens $x = (x_1, x_2, \ldots, x_n)$ with $x_t \in \mathcal{V}$. Let $\mathcal{C} = \{1, \ldots, K\}$ be a set of K class labels; here $K = 4$.

The task is to learn a function $f_\theta : \mathcal{V}^* \to \Delta^{K-1}$, mapping a token sequence of arbitrary length to a probability distribution over the K classes, where Δ^{K-1} denotes the probability simplex. Given a labelled training set $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, the parameters θ are chosen to minimise the categorical cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log f_\theta(x^{(i)})_{y^{(i)}}. \tag{3.1}$$

Both architectures studied here share a common three-stage structure:

1. an **embedding** stage mapping tokens to dense vectors,
2. a **contextualisation** stage producing position-wise representations that depend on the rest of the sequence — this is the stage in which the two architectures differ,
3. a **pooling and classification** stage reducing the variable-length sequence of representations to a single vector and projecting it to K logits.

Holding stages 1 and 3 as similar as possible while varying stage 2 is precisely the experimental design described in Chapter 4.

3.2 Embedding

An embedding matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ maps each token index to a row vector. For a sequence x of length n this yields

$$X = \big(E_{x_1}, E_{x_2}, \ldots, E_{x_n}\big)^\top \in \mathbb{R}^{n \times d}. \tag{3.2}$$

In this study E is randomly initialised and learned jointly with the rest of the model; no pretrained vectors are used. A designated padding index is held at the zero vector and excluded from gradient updates.

3.3 Recurrent networks and the vanishing gradient problem

3.3.1 The simple recurrent formulation

A simple recurrent network maintains a hidden state updated at each timestep:

$$h_t = \phi\big(W x_t + U h_{t-1} + b\big), \qquad h_0 = \mathbf{0}, \tag{3.3}$$

where ϕ is a saturating nonlinearity, typically tanh.

3.3.2 Backpropagation through time

Training requires the gradient of a loss $\mathcal{L}$ defined at the final timestep with respect to parameters that are shared across all timesteps. Applying the chain rule across the unrolled computation gives

$$\frac{\partial \mathcal{L}}{\partial h_k} = \frac{\partial \mathcal{L}}{\partial h_n} \prod_{t=k+1}^{n} \frac{\partial h_t}{\partial h_{t-1}}. \tag{3.4}$$

From Equation 3.3, each Jacobian factor is

$$\frac{\partial h_t}{\partial h_{t-1}} = U^{\top} \operatorname{diag}\big(\phi'(a_t)\big), \qquad a_t = W x_t + U h_{t-1} + b. \tag{3.5}$$

3.3.3 The pathology

Taking norms in Equation 3.4 and writing $\gamma = \sup_z |\phi'(z)|$ — which equals 1 for tanh and $\frac{1}{4}$ for the logistic sigmoid — gives the bound

$$\left\| \frac{\partial h_t}{\partial h_{t-1}} \right\| \;\le\; \gamma \, \|U\|, \qquad \text{hence} \qquad \left\| \prod_{t=k+1}^{n} \frac{\partial h_t}{\partial h_{t-1}} \right\| \;\le\; \big(\gamma \|U\|\big)^{\,n-k}. \tag{3.6}$$

The product is geometric in the distance $n-k$. If $\gamma\|U\| < 1$ the gradient decays exponentially and the network cannot assign credit across long spans — the *vanishing gradient* problem. If $\gamma\|U\| > 1$ it grows exponentially — the *exploding gradient* problem, which is the more tractable of the two and is routinely handled by gradient-norm clipping, as is done in this study.

Vanishing gradients cannot be addressed by clipping, because the information is destroyed rather than merely rescaled. The resolution requires an architectural change.

3.4 Long Short-Term Memory

3.4.1 Gating equations

LSTM introduces a cell state c_t alongside the hidden state h_t, together with three gates. At each timestep:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad \text{input gate} \tag{3.7}$$
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad \text{forget gate} \tag{3.8}$$
$$g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g) \quad \text{candidate cell} \tag{3.9}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad \text{output gate} \tag{3.10}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad \text{cell update} \tag{3.11}$$
$$h_t = o_t \odot \tanh(c_t) \quad \text{hidden output} \tag{3.12}$$

where σ is the logistic sigmoid and $\odot$ denotes elementwise multiplication.

Interpretively: the forget gate f_t decides what proportion of the existing cell contents to retain; the input gate i_t decides how much of the new candidate g_t to admit; and the output gate o_t decides how much of the resulting cell state to expose as the hidden output.

3.4.2 Why this solves the gradient problem

Consider the derivative of the cell state with respect to its own previous value. Differentiating Equation 3.11, and noting that f_t, i_t and g_t all depend on h_{t-1} rather than on c_{t-1} directly, the dominant term is

$$\frac{\partial c_t}{\partial c_{t-1}} \;=\; \operatorname{diag}(f_t) \;+\; (\text{terms via } h_{t-1}). \tag{3.13}$$

The leading term is multiplication by the forget gate, with *no weight matrix and no saturating nonlinearity in the path.* When $f_t \approx \mathbf{1}$, the gradient flows backwards across that timestep essentially unchanged. Over $n-k$ steps the decay is governed by $\prod_t f_t$ rather than by $(\gamma\|U\|)^{n-k}$, and since the forget gate is learned, the network can *choose* to keep this path open for exactly those features requiring long-range credit assignment. This is the constant error carousel of the original formulation [7], and it is the entire reason LSTM works.

3.4.3 Fused gate computation

Equations 3.7–3.10 share the same two inputs, x_t and h_{t-1}. In implementation the four gate pre-activations are therefore computed in a single pair of matrix products,

$$\begin{bmatrix} a_i \\ a_f \\ a_g \\ a_o \end{bmatrix} = \underbrace{W x_t}_{\in \mathbb{R}^{4d_h}} + \underbrace{U h_{t-1}}_{\in \mathbb{R}^{4d_h}} + b, \tag{3.14}$$

with $W \in \mathbb{R}^{4d_h \times d}$ and $U \in \mathbb{R}^{4d_h \times d_h}$, after which the result is split into four equal blocks. This is a pure efficiency transformation — it is mathematically identical to the separate form — but it is the form used in the implementation, and the *ordering* of the four blocks is a convention that must match the reference implementation for the verification in Chapter 6 to be meaningful.

3.4.4 Parameter count

An LSTM layer with input dimension d and hidden dimension d_h has

$$P_{\text{LSTM}} = 4\big(d\,d_h + d_h^2 + d_h\big) \tag{3.15}$$

parameters. A bidirectional layer doubles this.

3.5 Bidirectional recurrence

A bidirectional layer runs two independent recurrences,

$$\overrightarrow{h}_t = \text{LSTM}^{\rightarrow}(x_t, \overrightarrow{h}_{t-1}), \qquad \overleftarrow{h}_t = \text{LSTM}^{\leftarrow}(x_t, \overleftarrow{h}_{t+1}), \tag{3.16}$$

and concatenates them, $h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{2d_h}$. Each position is thereby contextualised by the whole sequence rather than only its prefix.

3.6 Pooling

Classification requires reducing $(h_1, \dots, h_n)$ to a single vector. Three strategies are standard.

Last state. Use h_n. For a unidirectional recurrent model this is the natural summary, but it is sensitive to the final token and discards earlier evidence except insofar as the state retained it.

Mean pooling. Use $\frac{1}{n}\sum_t h_t$. This weights all positions equally and is robust, but for a recurrent model it discards the privileged endpoint that the recurrence was constructed to produce.

Concatenation. Use $\big[\frac{1}{n}\sum_t h_t\,;\ \overrightarrow{h}_n\,;\ \overleftarrow{h}_1\big]$, combining both. This is the strategy adopted here, on the reasoning that the two summaries carry complementary information and documents of 39 tokens are short enough that the endpoint alone is noisy.

With padded batches, all pooling operations must be masked — a point of considerable practical importance developed in Section 3.7.4.

3.7 Attention

3.7.1 Scaled dot-product attention

Given queries $Q \in \mathbb{R}^{n\times d_k}$, keys $K \in \mathbb{R}^{m\times d_k}$ and values $V \in \mathbb{R}^{m\times d_v}$, scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V. \tag{3.17}$$

The matrix $QK^{\top} \in \mathbb{R}^{n\times m}$ contains a compatibility score for every query–key pair. The row-wise softmax converts each row into a probability distribution over the m positions, and the product with V forms, for each query, a convex combination of value vectors.

In *self*-attention, Q, K and V are all linear projections of the same input sequence, so $n = m$ and every position attends to every position including itself.

3.7.2 Why the $\sqrt{d_k}$ scaling is necessary

The scaling factor is not cosmetic. Suppose the components of q and k are independent with zero mean and unit variance. Then

$$\mathrm{E}[q \cdot k] = 0, \qquad \mathrm{Var}[q \cdot k] = \mathrm{Var}\left[\sum_{j=1}^{d_k} q_j k_j\right] = d_k. \tag{3.18}$$

The dot products therefore have standard deviation $\sqrt{d_k}$, growing with the head dimension. For large d_k the softmax receives inputs of large magnitude and saturates: its output approaches a one-hot vector, and its Jacobian approaches zero. Writing $p = \mathrm{softmax}(s)$, the Jacobian is

$$\frac{\partial p_i}{\partial s_j} = p_i\big(\delta_{ij} - p_j\big), \tag{3.19}$$

which vanishes as any $p_i \to 1$. Dividing by $\sqrt{d_k}$ restores unit variance to the scores and keeps the softmax in its responsive region, preserving gradient flow.

3.7.3 Multi-head attention

A single attention operation produces one weighted average per position, which constrains the model to one relational pattern per layer. Multi-head attention runs h attention operations in parallel on different learned projections:

$$\mathrm{head}_j = \mathrm{Attention}\Big(XW_j^Q,\, XW_j^K,\, XW_j^V\Big), \tag{3.20}$$

$$\mathrm{MHA}(X) = [\mathrm{head}_1; \ldots; \mathrm{head}_h]\, W^O. \tag{3.21}$$

With $d_k = d_v = d_{\text{model}}/h$, total computation is approximately that of a single head at full dimension, but the model can attend to several distinct relational patterns simultaneously.

In implementation the per-head projections are not separate matrices. A single projection of width d_{model} is computed and then *reshaped* into h blocks of width d_k. This reshape is the single most error-prone step in implementing attention, and it is the specific target of verification check 3 in Chapter 6.

3.7.4 Masking

Batched processing requires padding sequences to a common length. Padded positions must not influence the result. In attention this is enforced by adding a mask to the scores before the softmax:

$$s'_{ij} = \begin{cases} s_{ij} & \text{if position } j \text{ is a real token} \\ -\infty & \text{if position } j \text{ is padding,} \end{cases} \tag{3.22}$$

so that $\mathrm{softmax}(s')$ assigns exactly zero weight to padded keys.

In practice $-\infty$ is replaced by a large finite negative constant. The reason is numerical: if an entire row were masked, softmax of a vector of $-\infty$ evaluates to $0/0 = \mathrm{NaN}$, which propagates through the network and destroys the gradients. A large finite constant yields a harmless uniform distribution in that degenerate case.

3.8 Positional encoding

Self-attention as defined in Equation 3.17 is permutation-equivariant: permuting the input rows permutes the output rows identically, with no other change. A model built from self-attention alone therefore cannot distinguish word order at all — it would be a bag-of-words model with a sophisticated weighting scheme.

Position must consequently be injected explicitly. The sinusoidal scheme of Vaswani et al. [22] defines

$$PE_{(\mathrm{pos},\,2i)} = \sin\!\Big(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\Big), \tag{3.23}$$

$$PE_{(\mathrm{pos},\,2i+1)} = \cos\!\Big(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\Big), \tag{3.24}$$

and adds PE to the token embeddings. Each dimension is a sinusoid of a different wavelength, forming a geometric progression from 2π to $10000 \cdot 2\pi$. The scheme has two useful properties: it requires no parameters, and because $PE_{\mathrm{pos}+k}$ is a fixed linear function of PE_{pos} for any fixed offset k, relative position is linearly recoverable.

Token embeddings are scaled by $\sqrt{d_{\mathrm{model}}}$ before the positional signal is added, so that the two contributions are of comparable magnitude at initialisation.

3.9 Layer normalisation

Layer normalisation [1] standardises each position's feature vector independently:

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\varsigma^2 + \epsilon}} + \beta, \qquad \mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \qquad \varsigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2, \tag{3.25}$$

with learned $\gamma, \beta \in \mathbb{R}^d$ and a small ϵ for numerical stability.

Unlike batch normalisation, the statistics are computed over the feature dimension of a single example, so the operation is independent of batch composition and of sequence length. Note that ς^2 is the *biased* variance, normalising by d rather than $d-1$; this detail matters for the numerical agreement verified in Chapter 6.

3.10 Residual connections and block ordering

Each sublayer is wrapped in a residual connection, which provides an identity path for gradients. Two orderings are in use:

$$\text{post-norm:} \quad x \leftarrow \mathrm{LN}\big(x + \mathrm{Sublayer}(x)\big), \tag{3.26}$$

$$\text{pre-norm:} \quad x \leftarrow x + \mathrm{Sublayer}\big(\mathrm{LN}(x)\big). \tag{3.27}$$

In the post-norm arrangement the residual path passes through a normalisation at every layer, and at initialisation the expected gradient magnitude scales unfavourably with depth, which in practice necessitates a learning-rate warmup schedule. In the pre-norm arrangement the residual path is unobstructed from input to output, gradients are better conditioned at initialisation, and warmup becomes unnecessary [23]. The present study uses pre-norm, since

the training budget is a small number of epochs and a warmup schedule would consume a meaningful fraction of it.

The complete encoder block is

$$x \leftarrow x + \text{Dropout}\big(\text{MHA}(\text{LN}(x))\big), \tag{3.28}$$

$$x \leftarrow x + \text{Dropout}\big(W_2\ \text{GELU}(W_1\,\text{LN}(x))\big). \tag{3.29}$$

3.11 Complexity analysis

Table 3.1 summarises the structural comparison, following the analysis of Vaswani et al. [22]. Let n be sequence length and d the representation dimension.

Table 3.1: Per-layer complexity, sequential depth and maximum path length. *Sequential operations* counts steps that cannot be parallelised; *maximum path length* is the number of computational steps separating the two most distant positions.

Layer type	**Complexity per layer**	**Sequential operations**	**Max path length**
Recurrent (LSTM)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$

Three consequences follow, and all three are tested empirically in Chapter 7.

Path length. Self-attention connects any two positions in one step; recurrence requires $O(n)$ steps. This is the basis of the Transformer's advantage on long-range dependency. Its practical value is proportional to how much long-range structure the task actually contains.

Parallelism. Recurrence requires $O(n)$ strictly sequential steps; self-attention requires $O(1)$. On parallel hardware this is a decisive advantage in wall-clock terms.

Arithmetic. The comparison between $O(nd^2)$ and $O(n^2d)$ depends on the ratio n/d. Self-attention performs *less* total arithmetic only when $n < d$. With $n = 64$ and $d = 128$, as in this study, the two are of the same order, and neither dominates. This observation is central to the cost result reported in Section 7.6, and it is why that result does not contradict the theory.

Chapter 4

Dataset and Methodology

4.1 The AG News corpus

AG News is a topic classification benchmark derived from the AG corpus of news articles, introduced in its current form by Zhang et al. [24]. Each record consists of a class index, a headline and a short description. The four classes are balanced by construction.

Table 4.1: Composition of the AG News corpus as distributed.

Split	**Records**	**Per class**
Training file	120,000	30,000
Test file	7,600	1,900
Total	127,600	31,900

The corpus was obtained programmatically through the Kaggle API from the dataset `amananandrai/ag-news-cl` so that acquisition is reproducible rather than dependent on a manual download. The four class indices map to *World*, *Sports*, *Business* and *Science/Technology* respectively.

The text field used throughout is the concatenation of headline and description:

$$\text{text} = \text{Title} \; + \; \text{“.”} \; + \; \text{Description}.$$

4.2 Data cleaning

Inspection of the raw CSV files revealed a systematic artefact inherited from the original scrape. Escape sequences appear *literally* in the text rather than as the control characters they denote. A representative record reads:

```
Reuters - Short-sellers, Wall Street's dwindling\band of ultra-cynics, are seeing
green again.
```

Here `\b` is a literal backslash followed by the letter `b`, not a backspace. The phrase should read *“dwindling band of ultra-cynics”*.

This is not a cosmetic problem. Under the tokenisation scheme described in Section 4.4, the backslash is discarded as a non-matching character and the surrounding letters are welded into

the single token `dwindlingband`. Two genuine, frequent words are replaced by one spurious type that occurs almost nowhere else in the corpus and is therefore pruned by the minimum-frequency cutoff, mapping to the unknown token at both training and test time. The net effect is the silent destruction of lexical evidence in exactly the kind of content-bearing position that matters most for topic classification.

The cleaning step applies the regular expression `\\[a-z]`, replacing each such sequence with a single space, followed by whitespace normalisation. This is applied identically to both the training and test files, before any split is taken.

Methodological note. This artefact is easy to overlook, because a model trained on the uncleaned data still trains and still reports a plausible accuracy. The discovery came from reading the raw records directly rather than from any error signal. It is reported here because it is representative of a general principle in applied machine learning: a substantial fraction of achievable performance is determined before any model is defined.

4.3 Experimental design

4.3.1 The controlled subset

The case-study brief specifies the use of approximately 60,000 records for a controlled experiment. This was implemented as a stratified sample of exactly 15,000 records per class drawn from the 120,000-record training file with a fixed random seed, subsequently shuffled and split into training and validation partitions.

Table 4.2: Data splits used in the experiment. The test set is the full official file and is used once, for the final evaluation reported in Chapter 7.

Split	Records	Per class	Purpose
Training	54,000	13,500	parameter estimation
Validation	6,000	1,500	early stopping, learning-rate selection
Test	7,600	1,900	final evaluation only

All three partitions are exactly class-balanced, so accuracy and macro-averaged F_1 are directly interpretable and a majority-class baseline sits at 25%.

4.3.2 The four experimental controls

The comparison is designed so that architecture is the only free variable. Four quantities are held fixed.

1. **Data.** One subset, one split, one fixed seed. Both models see identical records in an identical order.
2. **Preprocessing.** One tokeniser, one vocabulary, one maximum sequence length. The integer tensors fed to the two models are not merely equivalent — they are the same objects.

3. **Capacity.** Both models are sized to approximately 265,000 non-embedding parameters, a difference of 0.19%. Without this control, any observed difference could be attributed to model size rather than architecture.

4. **Training budget.** Identical batch size, epoch cap, optimiser family, weight decay, gradient clipping, early-stopping criterion and random seed.

4.3.3 The one quantity that is not shared

The learning rate differs between the two models: 10^{-3} for the recurrent model and 3×10^{-4} for the Transformer. Both values were selected on the validation split and never on the test set.

This choice warrants explicit defence, since it appears to weaken the control. Imposing a single shared learning rate would *appear* fairer, but it would in fact handicap whichever architecture that particular value suited less well, and the experiment would then measure sensitivity to one arbitrary hyperparameter rather than the architectures themselves. Recurrent and self-attentive models are known to have different stable learning-rate regimes. The quantity that is properly controlled is the *compute budget*; the learning rate is a per-architecture tuning decision made on data that is not used for evaluation.

4.4 Preprocessing

4.4.1 Tokenisation

Documents are lowercased and tokenised with the regular expression `[a-z0-9']+`, which extracts maximal runs of alphanumeric characters and apostrophes. This retains contractions and possessives as single tokens while discarding punctuation.

The tokeniser is deliberately simple. A more sophisticated scheme — subword segmentation, for instance — would very likely improve both models. It is omitted precisely because both models must receive identical input: any tokeniser sophistication is a confound if it happens to suit one architecture better than the other. The simple scheme makes the comparison clean at a small cost in absolute accuracy.

4.4.2 Vocabulary

The vocabulary is constructed from the *training split only*. Neither the validation nor the test text contributes any token to the vocabulary, which eliminates a subtle and common form of information leakage.

Types occurring fewer than twice in the training split are discarded. Two special tokens are reserved: index 0 for padding and index 1 for unknown tokens. The resulting vocabulary contains **33,964** types.

4.4.3 Sequence length

Choosing a maximum sequence length is a trade-off between truncating information and wasting computation on padding. The choice was made from the measured distribution rather than by convention.

Table 4.3: Token-length statistics of the training split.

Statistic	**Tokens**
Median	39
Mean	39.4
95th percentile	58
Maximum	181
Chosen `MAX_LEN`	64
Coverage at this length	96.94%

Figure 4.1 shows the full distribution. A cutoff of 64 tokens leaves 96.94% of training documents untruncated while keeping the sequence dimension small enough for the recurrent model to remain tractable on CPU.

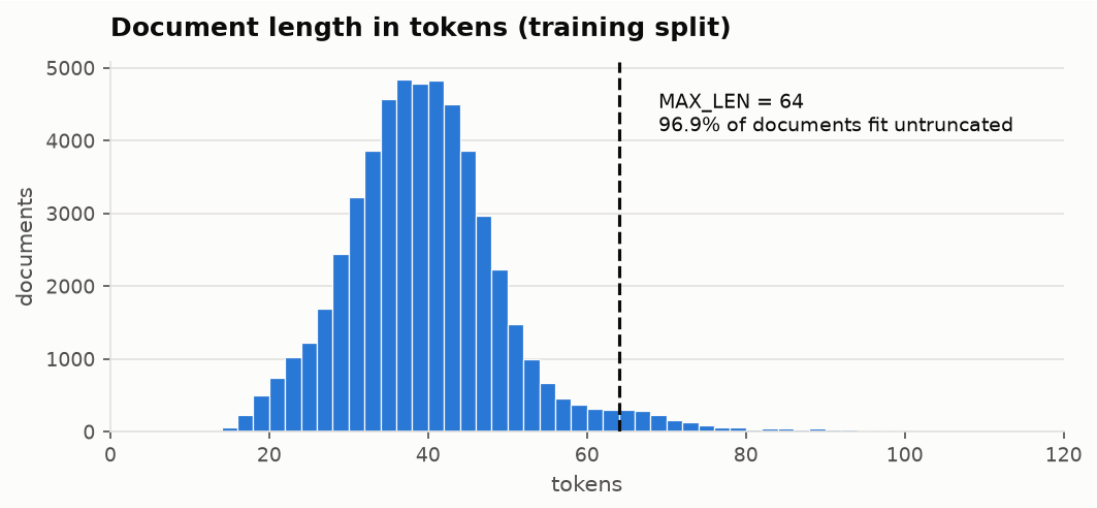


Figure 4.1: Distribution of document lengths in tokens on the training split. The dashed line marks the chosen cutoff of 64 tokens. The distribution is tightly concentrated around a median of 39 tokens — the property that, as argued in Chapter 8, accounts for the equivalence of the two architectures.

The central statistic of this study. The median document is 39 tokens long. Every substantive conclusion in Chapter 8 follows from this number.

4.4.4 Encoding

Each document is converted to a fixed-length vector of 64 integer indices, truncated or right-padded with the padding index as required, together with its true length. Documents reducing to zero tokens are mapped to a single unknown token so that no sequence has a zero-length mask.

4.5 Evaluation metrics

4.5.1 Accuracy and F_1

Since all splits are exactly balanced, accuracy is a meaningful primary metric. It is nonetheless reported alongside per-class precision, recall and F_1:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{4.1}$$

Macro-averaged F_1 — the unweighted mean of the per-class values — is used as the model selection criterion, because it weights all four classes equally and is sensitive to failure on any single class in a way that accuracy is not.

4.5.2 Confusion matrix

The 4×4 confusion matrix is reported row-normalised, so that entry (i, j) gives the proportion of true class i predicted as class j. With four classes there are twelve off-diagonal cells, and their structure is considerably more informative about model behaviour than any scalar summary.

4.5.3 McNemar's test

Comparing two classifiers on the same test set produces *paired* observations, and a test that respects the pairing is required. McNemar's test [14] considers only the discordant cases — documents on which the two models disagree — arranged as:

Table 4.4: Contingency structure for McNemar's test.

	Model B correct	**Model B wrong**
Model A correct	a	b
Model A wrong	c	d

The counts a and d — cases where both models agree — carry no information about which model is better and are discarded. Under the null hypothesis that the two models have equal error rates, each discordant case is equally likely to fall in cell b or cell c, so

$$b \sim \text{Binomial}(b + c,\ 0.5). \tag{4.2}$$

The exact binomial test on this statistic is used here in preference to the chi-squared approximation, since it is valid for any count.

Why this matters. Accuracy differences below one percentage point are routinely reported as improvements in the literature. On a 7,600-document test set, a difference of 0.09 percentage points corresponds to seven documents. Without a paired significance test there is no principled basis for distinguishing such a difference from sampling noise, and as Chapter 7 shows, in this case it cannot be so distinguished.

Table 4.5: Hardware and software environment. All timing results in Chapter 7 are specific to this configuration.

Component	**Specification**
Processor	12 logical cores, 6 PyTorch threads
Accelerator	none — CPU execution only
Memory	15 GB
Operating system	Linux (Fedora, kernel 6.17)
Python	3.14.8
PyTorch	2.15.0 (CPU build) [18]
Random seed	42, applied to `random`, `numpy` and `torch`

4.6 Experimental environment

The absence of a GPU is not incidental to the findings. It materially affects the cost comparison in Section 7.6, and the discussion there is explicit about which conclusions are hardware-dependent.

Chapter 5

Implementation

5.1 The from-scratch constraint

Both models are implemented from first principles. The precise meaning of that constraint is important, and is stated here before the code is presented.

PyTorch is used for three things: multidimensional array arithmetic, reverse-mode automatic differentiation, and the optimiser. The modules `nn.Linear`, `nn.Embedding` and `nn.Dropout` are used, but only as containers for learnable parameters — an `nn.Linear` is an affine map, not a sequence-modelling primitive.

Every operation specific to sequence modelling is written out. Table 5.1 lists the correspondence between the theory of Chapter 3 and the implementation.

Table 5.1: Components implemented from first principles, with the equations they realise.

Component	**Implemented as**	**Equations**
LSTM gating	`LSTMCellScratch`	3.7–3.12
Recurrence, bidirectional	`LSTMLayerScratch`	Sec. 3.3
Recurrent classifier	`BiLSTMClassifier`	Sec. 3.6
Positional encoding	`sinusoidal_positional_encoding`	Ch. 3
Layer normalisation	`LayerNormScratch`	Ch. 3
Scaled dot-product attention	`MultiHeadSelfAttentionScratch`	3.17
Multi-head split and merge	`MultiHeadSelfAttentionScratch`	3.21
Encoder block	`EncoderBlockScratch`	Ch. 3
Transformer classifier	`TransformerClassifier`	—

The following library primitives are **not used anywhere in either model**: `nn.LSTM`, `nn.GRU`, `nn.RNN`, `nn.MultiheadAttention`, `nn.TransformerEncoder`, `nn.TransformerEncoderLayer`, `nn.LayerNorm` and `F.scaled_dot_product_attention`. Two of these — `nn.LSTM` and `F.layer_norm` — appear exactly once each, inside the verification suite of Chapter 6, where they serve as numerical references.

5.2 Model A: the bidirectional LSTM

5.2.1 The cell

`LSTMCellScratch` implements Equations 3.7–3.12 directly. The four gate pre-activations are computed as a single fused product and split with `chunk(4)`.

```
class LSTMCellScratch(nn.Module):
    def __init__(self, d_in, d_hidden):
        super().__init__()
        self.d_hidden = d_hidden
        self.W = nn.Linear(d_in, 4 * d_hidden, bias=True)
        self.U = nn.Linear(d_hidden, 4 * d_hidden, bias=False)
        self.reset_parameters()

    def reset_parameters(self):
        k = 1.0 / math.sqrt(self.d_hidden)
        for p in self.parameters():
            nn.init.uniform_(p, -k, k)

    def forward(self, x_t, h_prev, c_prev):
        gates = self.W(x_t) + self.U(h_prev)        # (B, 4H) in one matmul
        i, f, g, o = gates.chunk(4, dim=-1)         # gate order: i, f, g, o
        i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
        g = torch.tanh(g)
        c_t = f * c_prev + i * g
        h_t = o * torch.tanh(c_t)
        return h_t, c_t
```

Two design decisions deserve comment.

Gate ordering. The chunks are ordered i, f, g, o. This ordering is arbitrary in principle — any consistent assignment trains equally well — but it is chosen to match PyTorch's internal convention exactly. That correspondence is what permits verification check 1 of Chapter 6 to copy the reference implementation's weight matrices directly into this cell and demand numerical equality. Choosing a different ordering would have made the strongest available correctness test impossible.

Bias placement. The reference implementation carries two bias vectors, one associated with each of the input and hidden projections; their sum is mathematically equivalent to a single bias. This implementation uses one bias on `W` and none on `U`. The verification check accounts for this by copying the *sum* of the two reference biases.

5.2.2 The recurrence, and the masking problem

`LSTMLayerScratch` unrolls the cell over the time dimension. Because batched sequences are padded to a common length, this is the point at which a from-scratch implementation is most likely to go subtly wrong.

```
class LSTMLayerScratch(nn.Module):
    def __init__(self, d_in, d_hidden, reverse=False):
        super().__init__()
        self.cell = LSTMCellScratch(d_in, d_hidden)
        self.d_hidden, self.reverse = d_hidden, reverse
```

```
    def forward(self, x, mask):
        B, T, _ = x.shape
        h = x.new_zeros(B, self.d_hidden)
        c = x.new_zeros(B, self.d_hidden)
        steps = range(T - 1, -1, -1) if self.reverse else range(T)
        outputs = []
        for t in steps:
            m = mask[:, t].unsqueeze(1)                    # (B, 1)
            h_new, c_new = self.cell(x[:, t], h, c)
            h = m * h_new + (1.0 - m) * h                  # padded step = no-op
            c = m * c_new + (1.0 - m) * c
            outputs.append(h)
        if self.reverse:
            outputs.reverse()                              # back to position order
        return torch.stack(outputs, dim=1), h
```

Problem 1: state corruption by padding. The naive implementation applies the cell at every timestep unconditionally. Padded positions then continue to drive the recurrence: the hidden state at the nominal end of the sequence reflects an arbitrary number of pad-token updates rather than the true end of the document. Because padding length varies across a batch, the magnitude of this corruption varies per example.

The fix is to gate the state update by the mask,

$$h_t = m_t \odot \tilde{h}_t + (1 - m_t) \odot h_{t-1}, \tag{5.1}$$

and identically for c_t, where $m_t \in \{0, 1\}$ indicates a real token. A padded step is then an exact no-op: the state is carried forward unchanged.

Problem 2: the backward direction's starting point. The backward recurrence must begin at each sequence's *own* final token, not at index 63. Sequences in a batch have different true lengths, so there is no single starting index.

The implementation resolves this without any per-example index arithmetic, by exploiting the masking just described. Padding sits at the right of each sequence, and masked steps are exact no-ops. Iterating from $t = T - 1$ down to $t = 0$ therefore passes through the padded region performing no updates, and the recurrence effectively begins at each sequence's last real token. The returned final state is the correct per-sequence endpoint in both directions.

This equivalence is an argument, not an observation, and arguments of this kind are exactly what should be checked empirically. Verification checks 6 and 7 do so.

5.2.3 The classifier

```
class BiLSTMClassifier(nn.Module):
    def forward(self, ids, lengths=None):
        mask = (ids != PAD_ID).float()
        x = self.embedding(ids)
        out_f, h_f = self.forward_layer(x, mask)
        out_b, h_b = self.backward_layer(x, mask)
        seq = torch.cat([out_f, out_b], dim=-1)               # (B, T, 2H)
        denom = mask.sum(dim=1, keepdim=True).clamp(min=1.0)
        mean_pooled = (seq * mask.unsqueeze(-1)).sum(dim=1) / denom
        features = torch.cat([mean_pooled, h_f, h_b], dim=-1) # (B, 4H)
        return self.classifier(self.dropout(features))
```

The pooling follows Section 3.6: the masked mean over all positions is concatenated with the final state of each direction, giving $4d_h$ features. Note that the mean is computed with an explicitly masked numerator

and a denominator equal to the true length — dividing by the padded length would systematically shrink the representation of short documents.

5.3 Model B: the Transformer encoder

5.3.1 Positional encoding and layer normalisation

```
def sinusoidal_positional_encoding(max_len, d_model):
    position = torch.arange(max_len, dtype=torch.float32).unsqueeze(1)
    even_i   = torch.arange(0, d_model, 2, dtype=torch.float32)
    div_term = torch.exp(-math.log(10000.0) * even_i / d_model)
    pe = torch.zeros(max_len, d_model)
    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)
    return pe

class LayerNormScratch(nn.Module):
    def forward(self, x):
        mu  = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        return self.gamma * (x - mu) / torch.sqrt(var + self.eps) + self.beta
```

The `unbiased=False` argument is not incidental. The reference implementation normalises by d rather than $d-1$, and using the unbiased estimator here would produce a small but systematic discrepancy. Verification check 2 confirms agreement to 5×10^{-7}.

5.3.2 Multi-head self-attention

```
class MultiHeadSelfAttentionScratch(nn.Module):
    def _split_heads(self, t):
        B, T, _ = t.shape
        return t.view(B, T, self.n_heads, self.d_k).transpose(1, 2)   # (B,h,T,d_k)

    def forward(self, x, key_mask=None, return_attention=False):
        B, T, _ = x.shape
        q = self._split_heads(self.W_q(x))
        k = self._split_heads(self.W_k(x))
        v = self._split_heads(self.W_v(x))

        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.d_k)      # (B,h,T,T)
        if key_mask is not None:
            scores = scores.masked_fill(key_mask[:, None, None, :] == 0, -1e9)

        attention = torch.softmax(scores, dim=-1)
        context = self.dropout(attention) @ v
        context = context.transpose(1, 2).reshape(B, T, self.n_heads * self.d_k)
        return self.W_o(context)
```

Three points are worth drawing out.

The head reshape. The projections are computed once at full width and then reshaped into h heads. The sequence `view` then `transpose` is compact but opaque: it is easy to write a variant that runs without error while silently interleaving the head dimensions, which would mix features across heads.

Verification check 3 targets this specifically by comparing against a reference that slices head columns explicitly.

The mask broadcast. The mask has shape (B, T) and is indexed to $(B, 1, 1, T)$, broadcasting across heads and across query positions. The semantics are that every query, in every head, ignores padded *keys*.

The finite mask constant. The mask is applied as -10^9 rather than $-\infty$, for the numerical reason given in Section 3.7.4: a fully-masked row would otherwise produce `NaN` and destroy the gradients of the entire batch.

5.3.3 Encoder block and classifier

```
class EncoderBlockScratch(nn.Module):
    def forward(self, x, key_mask=None):
        x = x + self.dropout(self.attention(self.norm1(x), key_mask))
        h = self.norm2(x)
        h = self.ff_out(self.dropout(F.gelu(self.ff_in(h))))
        return x + self.dropout(h)
```

The pre-norm ordering is visible directly: normalisation is applied to the *input* of each sublayer, and the residual stream itself is never normalised, leaving an unobstructed gradient path from output to input.

The classifier embeds, scales by $\sqrt{d_{\text{model}}}$, adds the positional encoding, applies the encoder blocks, applies a final normalisation, and reduces to a single vector by masked mean pooling before the linear head.

5.4 Training harness

Both models are trained by a single function, so that the training budget is identical by construction rather than by careful duplication.

Table 5.2: Training configuration. All values except the learning rate are shared.

Setting	**BiLSTM**	**Transformer**
Optimiser	AdamW [10, 12]	
Weight decay	0.01	
Batch size	128	
Maximum epochs	8	
Gradient clipping	1.0 (global norm)	
Early stopping	validation macro-F_1, patience 2	
Checkpoint selection	best validation macro-F_1	
Random seed	42	
Learning rate	10^{-3}	3×10^{-4}

Gradient clipping at unit global norm addresses the exploding-gradient half of the pathology analysed in Section 3.3. Early stopping on macro-F_1 rather than accuracy ensures that a model degrading on a single class is not selected.

5.5 Architecture configuration and parameter budget

Table 5.3: Architectural hyperparameters.

BiLSTM		**Transformer**	
Embedding dimension	128	Model dimension d_{model}	128
Hidden dimension	128	Attention heads h	4
Directions	2	Encoder layers	2
Pooled features	512	Feed-forward dimension d_{ff}	256
Dropout	0.30	Dropout	0.10

These sizes were chosen to equalise non-embedding capacity, which is the quantity that must be controlled for the comparison to be about architecture.

Table 5.4: Parameter budget. The embedding table is identical in size for both models and dominates each total; the non-embedding column is therefore the meaningful measure of architectural capacity.

Model	**Total**	**Embedding**	**Non-embedding**
BiLSTM	4,612,612	4,347,392	265,220
Transformer	4,613,124	4,347,392	265,732
Difference	512	0	512 (0.19%)

The embedding table accounts for approximately 94% of each model's parameters. Reporting only total parameter counts would therefore obscure the comparison almost entirely — the two models would appear to differ by 0.01%, which is true but uninformative. The non-embedding figures differ by 512 parameters out of roughly 265,000, which is close enough that no observed performance difference can reasonably be attributed to capacity.

Chapter 6

Verification of the Hand-Written Layers

6.1 Why verification is methodologically necessary

This chapter is placed before the results, rather than in an appendix, because its content is a precondition for those results meaning anything.

The argument is as follows. Consider a hand-written LSTM cell in which the input and forget gates have been transposed — a single-character error in the order of a tuple unpacking. The resulting model is not broken in any observable sense. It compiles, it trains, its loss decreases, and it reports a test accuracy perhaps two or three points below what a correct implementation would achieve. Nothing in the output distinguishes it from a correct implementation of a slightly weaker architecture.

Now consider the purpose of this report: to compare two architectures and to interpret the resulting difference. A three-point degradation introduced by an implementation defect in one of the two models is of precisely the magnitude that would be reported, in good faith, as an architectural finding. The experiment would produce a confident, well-presented and entirely spurious conclusion.

The characteristic failure mode of from-scratch numerical code is silent degradation, not an observable error. Testing is therefore not a matter of engineering hygiene here; it is what licenses the inferences in Chapter 7.

Nine checks are specified below. All are executed on every run of the notebook, before any training occurs, and the suite raises an assertion on failure rather than emitting a warning, so that it is not possible to obtain results from an unverified implementation.

6.2 The reference implementations

Two library primitives are used in this chapter and nowhere else in the project: `nn.LSTM` and `F.layer_norm`. Their use here is legitimate, and indeed necessary: the strongest available evidence that a hand-written layer is correct is that it agrees numerically with an independent, widely-tested implementation of the same mathematics.

The distinction being drawn is between *using* a primitive to build a model — which the from-scratch constraint forbids — and *comparing against* one to verify a model, which is the only rigorous way to discharge that constraint.

6.3 Checks 1–2: equivalence against reference implementations

6.3.1 Check 1 — the LSTM cell

This is the strongest check in the suite. Rather than testing that the hand-written cell behaves *similarly* to a reference, it tests that it is *identical* given identical parameters.

The procedure copies the reference implementation's input-projection matrix, its hidden-projection matrix, and the *sum* of its two bias vectors into the hand-written cell, then runs both over the same random input sequence and compares every timestep.

```
ref_lstm = nn.LSTM(D, H, num_layers=1, batch_first=True)   # reference only
ours = LSTMLayerScratch(D, H, reverse=False)
with torch.no_grad():
    ours.cell.W.weight.copy_(ref_lstm.weight_ih_l0)
    ours.cell.W.bias.copy_(ref_lstm.bias_ih_l0 + ref_lstm.bias_hh_l0)
    ours.cell.U.weight.copy_(ref_lstm.weight_hh_l0)
ref_out, (ref_h, _) = ref_lstm(x)
our_out, our_h = ours(x, full_mask)
```

Observed maximum absolute deviation: 5.96×10^{-8} across all timestep outputs and 4.47×10^{-8} on the final hidden state. Both are at the level of single-precision rounding.

This result simultaneously confirms the gate algebra of Equations 3.7–3.12, the gate *ordering* convention, the handling of the fused bias, and the correctness of the unrolled recurrence. A transposition of any two gates would produce a deviation of order unity.

6.3.2 Check 2 — layer normalisation

The hand-written normalisation is compared against `F.layer_norm` with the same learned parameters and a deliberately non-trivial input distribution (mean 1.5, standard deviation 3.0), so that an error in either centring or scaling would be visible.

Observed maximum absolute deviation: 4.77×10^{-7}. This confirms in particular the use of the biased variance estimator; substituting the unbiased estimator produces a deviation several orders of magnitude larger.

6.4 Check 3: attention against an independent formulation

The head split and merge is the most error-prone part of a multi-head attention implementation, because the reshaping operations are compact and their failure mode is a silent permutation of features rather than an exception.

A naive reference is therefore computed, which processes one *(batch, head)* pair at a time and slices head columns explicitly:

```
for b in range(x.size(0)):
    head_outputs = []
    for h in range(n_heads):
        sl = slice(h * d_k, (h + 1) * d_k)
        qh, kh, vh = q_all[b, :, sl], k_all[b, :, sl], v_all[b, :, sl]
        s = (qh @ kh.T) / math.sqrt(d_k)
        s = s.masked_fill(~km[b].unsqueeze(0), -1e9)
        head_outputs.append(torch.softmax(s, dim=-1) @ vh)
    naive[b] = mha.W_o(torch.cat(head_outputs, dim=-1))
```

Why the reference is formulated this way. The property under test is that `view(B,T,h,d_k).transpose(1,2)` corresponds to contiguous column blocks of the projection. A reference written with `einsum` would be more elegant, but it would share the very assumption being tested and would therefore prove nothing. Explicit column slicing with a Python loop is deliberately the least clever formulation available, and that is its merit: it has no assumptions in common with the implementation.

Observed maximum absolute deviation: 8.94×10^{-8}.

6.5 Checks 4–5: mathematical invariants

Checks 1–3 compare against an alternative implementation. Checks 4–5 instead assert properties that any correct implementation must satisfy, independent of how it is written.

Check 4 — attention rows sum to one. Each row of the attention matrix is a probability distribution over positions. Observed maximum deviation from unity: 1.79×10^{-7}. This would fail immediately if the softmax were applied over the wrong axis — an error that is otherwise hard to detect, since the result remains dimensionally valid and the model still trains.

Check 5 — padded keys receive no attention mass. The maximum attention weight assigned to any padded key, over all queries and heads, is 0.00×10^{0} — exactly zero in floating point, as the masked logits underflow the softmax completely. This confirms the mask reaches the scores correctly and is broadcast over the intended axes.

6.6 Checks 6–7: padding invariance

These are the decisive checks of the suite, and they test the masking arguments made in Section 3.7.4 and in Chapter 5.

The property. Appending additional padding tokens to a batch must not change either model's output. Padding is an artefact of batching and carries no information; if logits move when padding is added, the model is reading the artefact.

The test constructs a batch with mixed true lengths, appends twenty further padding positions, and compares logits:

```
short_ids = torch.randint(2, 500, (6, 12))
short_ids[0, 8:] = PAD_ID          # mixed real lengths within the batch
short_ids[3, 5:] = PAD_ID
padded_ids = torch.cat([short_ids, torch.full((6, 20), PAD_ID)], dim=1)
d = (model(short_ids) - model(padded_ids)).abs().max()
```

Observed deviations: 0.00×10^{0} for the recurrent model — exact, as the masked update is an algebraic identity — and 1.19×10^{-7} for the Transformer, at the level of floating-point reassociation.

Why these checks carry the most weight. The two masking defects described in Chapter 5 — unmasked state updates and a backward pass beginning at the wrong position — produce no error, no warning and no visible symptom. They degrade accuracy by an amount that varies with the padding distribution of the batch, which is to say, invisibly. Padding invariance is the only practical means of detecting them, and the exact-zero result for the recurrent model confirms that the argument given in Chapter 5 — that reverse iteration under masking is equivalent to starting at each sequence's true endpoint — is sound in implementation as well as in principle.

6.7 Checks 8–9: gradient viability

The preceding checks all test the forward computation. A model may have a perfectly correct forward pass and still fail to learn, if gradients do not reach its parameters.

Checks 8 and 9 take a 200-example subset and train each model on it for 30 optimisation steps, requiring training accuracy above 95%. A model with a severed gradient path, a detached tensor or a vanishing initialisation cannot satisfy this.

Both models reach **100%** training accuracy within the step budget.

This check also serves a secondary purpose: a model that cannot overfit 200 examples has insufficient capacity or an optimisation problem, either of which would need to be resolved before any comparison of generalisation is meaningful.

6.8 Results

Table 6.1: Results of the nine verification checks. All pass. Tolerances are 10^{-5} for equivalence checks, 10^{-6} for layer normalisation and 10^{-4} for padding invariance.

#	Check	Measured	Result
1	LSTM cell vs. reference	5.96×10^{-8}	pass
2	Layer norm vs. reference	4.77×10^{-7}	pass
3	Attention vs. naive per-head loop	8.94×10^{-8}	pass
4	Attention rows sum to one	1.79×10^{-7}	pass
5	Padded keys receive zero mass	0.00×10^{0}	pass
6	BiLSTM padding invariance	0.00×10^{0}	pass
7	Transformer padding invariance	1.19×10^{-7}	pass
8	BiLSTM overfits 200 examples	100% / 30 steps	pass
9	Transformer overfits 200 examples	100% / 30 steps	pass

All nine checks pass. The implementations are therefore numerically faithful to the mathematics of Chapter 3, and the experimental results that follow can be attributed to the architectures rather than to implementation defects.

6.9 What this suite does not establish

Intellectual honesty requires stating the limits of the verification.

The suite establishes that the implemented layers compute the intended functions. It does *not* establish that the surrounding experimental pipeline is free of error: a defect in the data splitting, the label mapping or the metric computation would not be detected by any of these nine checks. Nor does it establish that the chosen architectures are well suited to the task, which is an empirical question answered in Chapter 7.

The balanced-accuracy sanity check — both models landing in the 90–92% band predicted by the literature survey in Table 2.1 — provides weak additional evidence that the pipeline as a whole is sound, but it is corroboration rather than proof.

Chapter 7

Results

All results in this chapter are computed on the full official 7,600-record AG News test set, from each model's restored best-validation checkpoint. The test set was used once.

7.1 Training dynamics

Table 7.1: Training summary. Both models stopped early, well inside the epoch cap of eight.

Quantity	**BiLSTM**	**Transformer**
Learning rate	10^{-3}	3×10^{-4}
Epochs run	3	5
Best epoch	1	3
Best validation macro-F_1	0.9091	0.9077
Total training time (s)	126	353

Figure 7.1 shows the learning curves. Two observations follow, and the first is reported because it qualifies the comparison rather than because it flatters it.

Both models overfit rapidly, and the recurrent model peaked after a single epoch. Training loss continues to fall throughout, while validation loss rises from epoch 2 onward in both cases. The early-stopping criterion detected this and restored the correct checkpoint, but the consequence is that the recurrent model's *effective* training budget was one epoch of 422 optimisation steps. The epoch cap of eight was never the binding constraint for either model.

The Transformer converges more slowly but more smoothly. It reaches its optimum at epoch 3 rather than epoch 1, consistent with its lower learning rate, and its validation macro-F_1 curve is less abrupt. Neither model benefits from further training under this configuration.

7.2 Headline results

Both models land in the 90–92% band anticipated by the survey of published results in Table 2.1, which is the expected range for a from-scratch model without pretraining. The recurrent model is nominally ahead on every metric except test loss.

This difference must not be interpreted before it is tested. A gap of 0.09 percentage points on 7,600 documents is seven documents. Section 7.5 determines whether that is distinguishable from

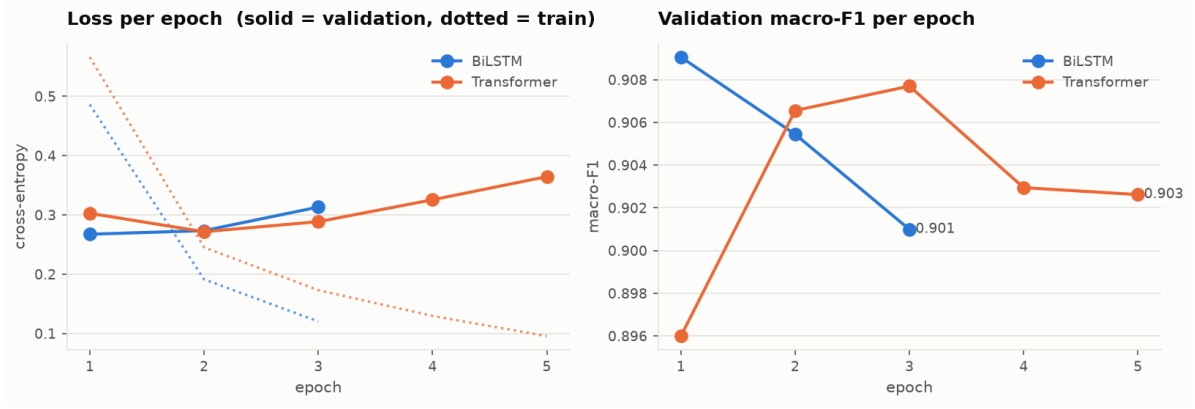


Figure 7.1: Learning curves. Left: cross-entropy per epoch, solid for validation and dotted for training. Right: validation macro-F_1. The divergence between training and validation loss from epoch 2 is the overfitting referred to in the text.

Table 7.2: Headline results on the held-out test set. The accuracy difference is 0.09 percentage points, corresponding to seven documents out of 7,600.

Model	Accuracy	Macro F_1	Weighted F_1	Test loss
BiLSTM	0.9114	0.9111	0.9111	0.2797
Transformer	0.9105	0.9104	0.9104	0.2944
Difference	0.0009	0.0007	0.0007	−0.0147

noise. It is not.

7.3 Per-class performance

Table 7.3: Per-class precision, recall and F_1. Support is 1,900 documents for every class.

	BiLSTM			Transformer		
Class	P	R	F_1	P	R	F_1
World	0.9366	0.8789	0.9069	0.9227	0.9053	0.9139
Sports	0.9459	0.9853	0.9652	0.9553	0.9674	0.9613
Business	0.8850	0.8705	0.8777	0.8889	0.8674	0.8780
Sci/Tech	0.8791	0.9111	0.8948	0.8754	0.9021	0.8885
Macro avg	0.9117	0.9114	0.9111	0.9106	0.9105	0.9104

The striking feature of Table 7.3 is not the magnitude of any difference but the *similarity of the profile*. Both models order the classes identically by difficulty:

$$\text{Sports } (\approx 0.96) \;>\; \text{World } (\approx 0.91) \;>\; \text{Sci/Tech } (\approx 0.89) \;>\; \text{Business } (\approx 0.878)$$

On Business the two models agree to four decimal places (0.8777 versus 0.8780). The two architectures do not merely achieve the same aggregate accuracy; they find the same classes easy and the same classes

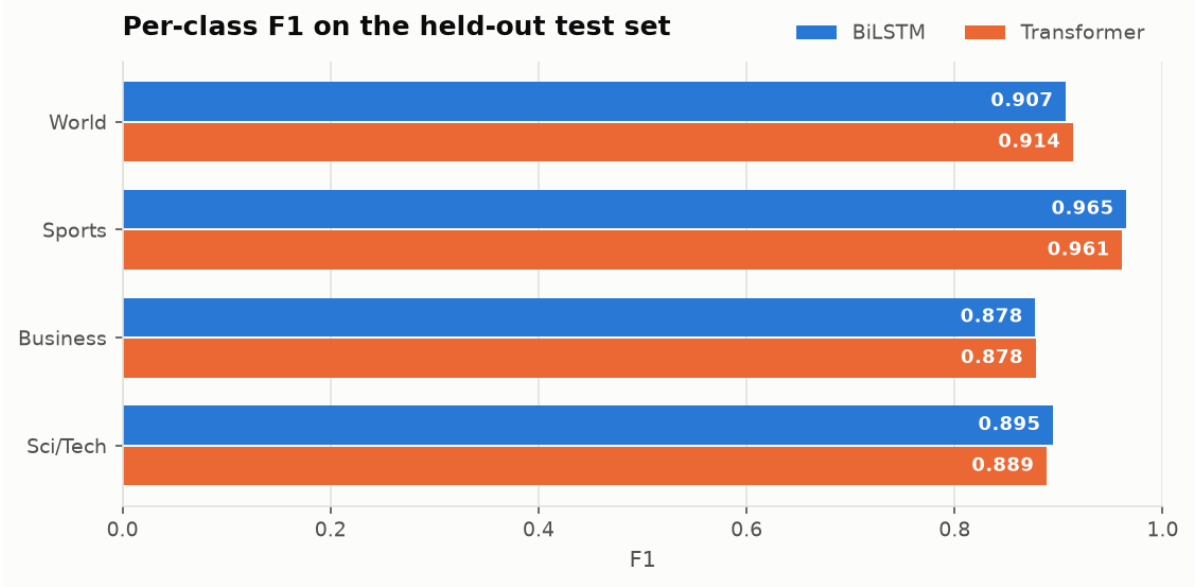


Figure 7.2: Per-class F_1 for both models. The difficulty ordering is identical: Sports is easiest for both, Business hardest for both.

hard, to a degree that suggests both are limited by the same property of the data rather than by their own representational capacity.

7.4 Confusion structure

Table 7.4: Largest off-diagonal confusions, as a proportion of the true class.

True	Predicted	BiLSTM	Transformer
Business	Sci/Tech	9.2% (174)	9.6% (183)
World	Business	5.4% (103)	4.1% (77)
Sci/Tech	Business	5.3% (101)	5.3% (101)
World	Sports	3.7% (71)	2.6% (49)

The structure is consistent across both models. The dominant error is Business misclassified as Sci/Tech, and its converse is the third largest. Together, the Business–Sci/Tech boundary accounts for roughly a quarter of all errors made by either model.

Sports is nearly separable from everything else — 98.5% and 96.7% recall — which is unsurprising given its highly distinctive vocabulary. The difficulty is concentrated entirely in the commercial and technological categories, whose vocabularies overlap substantially.

7.5 Statistical significance

Applying McNemar's test as specified in Section 4.5.3 to the paired predictions:

The models agree on 6,740 + 493 = 7,233 documents, or **94.78%** of the test set. The discordant set contains 367 documents: 187 answered correctly only by the recurrent model and 180 only by the Transformer.

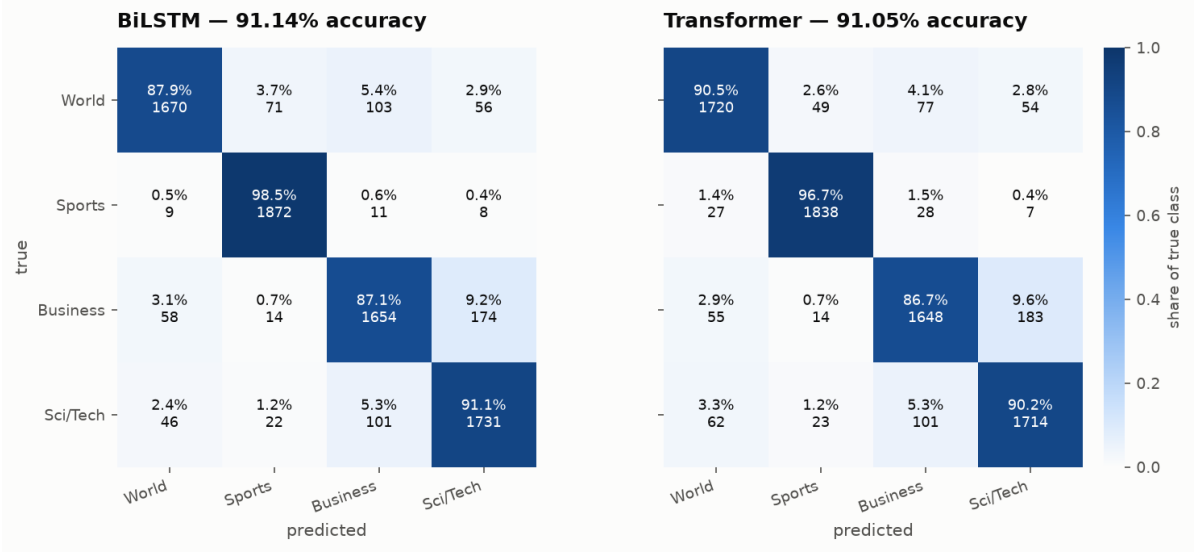


Figure 7.3: Row-normalised confusion matrices on a shared colour scale. Each row sums to 100% of the true class. The dominant off-diagonal cell is Business predicted as Sci/Tech for both models.

Table 7.5: McNemar contingency table on the 7,600 test documents.

	BiLSTM correct	**BiLSTM wrong**
Transformer correct	6,740	180
Transformer wrong	187	493

Under the null hypothesis of equal error rates, the number falling to either model follows Binomial$(367, 0.5)$. The exact binomial test gives

$$p = 0.754. \tag{7.1}$$

The difference is not statistically significant at $\alpha = 0.05$**.** A split of 187 against 180 is as close to an even division as a test set of this size can resolve. The correct conclusion is that **the two architectures are indistinguishable in accuracy on this task** – not that the recurrent model is marginally superior.

This is the central result of the study, and it would have been inaccessible from the headline accuracy figures alone. A 0.09 percentage point gap presented without a significance test invites exactly the over-interpretation that the test forecloses.

7.6 Computational cost

At matched capacity and matched accuracy, the recurrent model trains **1.67 times faster per epoch** and performs inference **2.43 times faster**.

This is the opposite of the expected result, and it requires careful interpretation rather than explanation away.

The standard argument, formalised in Table 3.1, is that recurrence requires $O(n)$ strictly sequential steps whereas self-attention requires $O(1)$, so attention should be faster in wall-clock terms. That argument is correct, but it is an argument about *parallelisability*, and it is conditional on two things that do not hold here.

Table 7.6: Computational cost on the CPU-only environment of Table 4.5.

Measure	**BiLSTM**	**Transformer**
Non-embedding parameters	265,220	265,732
Seconds per epoch	41.2	68.6
Total training time (s)	126	353
Inference (ms per 1,000 documents)	138	335
Test accuracy	0.9114	0.9105

First, **parallel hardware**. The advantage of $O(1)$ sequential depth is realised only when there are parallel resources to exploit. On six CPU threads there is very little parallelism available, so the Transformer cannot convert its structural advantage into elapsed time.

Second, **the relationship between n and d.** Per-layer arithmetic is $O(nd^2)$ for recurrence and $O(n^2d)$ for self-attention. Self-attention performs less total arithmetic only when $n < d$. Here $n = 64$ and $d = 128$, so the two are of the same order and neither dominates. The Transformer performs more total floating-point work per document — two encoder blocks of four-head attention over all 64 positions, each with a 256-wide feed-forward sublayer — while the recurrent model performs less work spread over 64 sequential dispatches.

When neither parallelism nor arithmetic favours attention, the constant factors decide, and here they favour recurrence.

What this result does and does not license. It does not license the claim that recurrent networks are faster than Transformers. It licenses the narrower and more useful claim that *at short sequence lengths on sequential hardware, the Transformer's architectural advantage does not materialise in wall-clock terms.* The recurrent model's disadvantage grows linearly with sequence length, and on parallel hardware the ordering would be expected to reverse. This is a finding about regime, not about architecture, and it is actionable precisely for that reason.

7.7 Qualitative error analysis

7.7.1 Documents both models misclassify

Of the 7,600 test documents, 493 are misclassified by both models. Table 7.7 gives a representative sample.

In every case the prediction is defensible. A story about United Nations funding to control locust swarms is an international affairs story *and* an agricultural science story. A telecoms merger is a business story *and* a technology story. These are not model failures in any useful sense; they are documents whose single gold label is one of several admissible choices.

This is the practical upper bound on achievable accuracy for the task, and it explains why the published results in Table 2.1 compress into such a narrow band.

7.7.2 Documents on which the models disagree

The disagreements are instructive. Several documents contain strong lexical evidence for two classes simultaneously — *Formula One* alongside *multi-million-dollar*, or *banks* and *High Court* alongside *F1* — and the two models resolve the conflict differently. Notably, neither model is systematically correct in these cases: the first and third examples show the models trading places on the same underlying ambiguity.

This is consistent with the statistical finding of Section 7.5. The disagreements are not evidence of

Table 7.7: Representative documents misclassified by both models. In each case the predicted label is defensible.

True	**Both predicted**	**Document (truncated)**
Sci/Tech	World	U.N. Urges Funds to Curb African Locusts — with swarms threatening crops in a number of African countries …
World	Sports	Argentina Basketball Coach Magnano Quits — Ruben Magnano, who coached Argentina to the Olympic gold medal …
Business	Sci/Tech	SBC links e-mail, voice messages, fax — SBC announced a new service that integrates voice messages, faxes …
Business	World	Nepal blockade 'blow to tourism' — Nepal tour operators say tourists cancelled millions of dollars of bookings …
Sci/Tech	World	Singapore Telecom ties up with Malaysia's TIME dotcom — SingTel said it has …

a systematic difference in capability; they are what one would expect from two comparably-capable models resolving genuinely ambiguous cases with different random tie-breaking.

Table 7.8: Documents where the two models differ. Several turn on a single lexical cue.

True	**BiLSTM**	**Transf.**	**Document (truncated)**
Sports	Sports	Business	Cape Town F1 bid — plan to build a multi-million-dollar race track …
World	Business	World	ConocoPhillips boosts LUKoil stake to 10 percent …
Business	Sports	Business	Banks seek court ruling over F1 — three banks go to the High Court in London …
Sports	Sports	World	New Namath Book Is Fact, Not Fiction — if you read the recent excerpt in Sports Illustrated …
World	World	Business	Australia: Wallets trump war — a majority oppose the Iraq war, but were more concerned …

Chapter 8

Discussion

8.1 Answering the research question

The research question was: *how do recurrent networks and Transformers compare at classifying short news texts?* The evidence of Chapter 7 supports a four-part answer.

8.1.1 They are indistinguishable in accuracy

The measured difference is 0.09 percentage points. McNemar's exact test on the paired predictions gives $p = 0.754$, the models agree on 94.78% of test documents, and the discordant set splits 187 to 180. There is no basis in this evidence for preferring either architecture on accuracy grounds.

This is a negative result, and it is reported as the principal finding rather than buried. Negative results of this kind are underreported in the literature, with the consequence that accuracy differences well inside the noise floor are routinely presented as architectural improvements. The discipline of applying a paired significance test is what converts an apparent 0.09-point win into the correct conclusion of equivalence.

8.1.2 The explanation is document length

The equivalence is not a coincidence, and it is not evidence that self-attention is overrated in general. It follows from a specific property of the data.

Self-attention's structural advantage, established in Table 3.1, is a maximum path length of $O(1)$ between any two positions, against $O(n)$ for recurrence. This is an advantage in modelling *long-range* dependency.

The median AG News document is 39 tokens long. In a 39-token document there are essentially no long-range dependencies to model. The furthest two tokens can be is a few dozen positions, which a bidirectional LSTM traverses without difficulty — this is precisely the regime the gating mechanism of Section 3.3 was designed to handle, and it is two orders of magnitude short of where the gating mechanism begins to fail.

Furthermore, the signal in topic classification is overwhelmingly lexical. The presence of *coach*, *Olympic* and *medal* identifies a sports story regardless of their arrangement. This observation is corroborated independently by the literature: FastText, which averages word vectors and models no order whatsoever, reaches approximately 92.5% on this benchmark (Table 2.1) — at or above both models in this study. If a bag-of-embeddings model is competitive, then the marginal value of *any* sequence modelling on this task is small, and the marginal value of one sequence model over another is smaller still.

The general form of the argument. An architecture's advantage can only be realised on a task that

exercises the property the architecture is good at. Transformers are superior at modelling long-range dependency; AG News contains almost none; therefore the two architectures perform equivalently. This is not a limitation of the Transformer but a statement about the task.

8.1.3 They fail in the same way

The equivalence runs deeper than aggregate accuracy. The two models share:

- an identical per-class difficulty ordering (Sports, World, Sci/Tech, Business),
- near-identical per-class F_1, agreeing to four decimal places on Business,
- the same dominant confusion, Business predicted as Sci/Tech, at 9.2% and 9.6%,
- agreement on 94.78% of individual test predictions.

Two models with materially different inductive biases would be expected to differ somewhere in this profile even at equal overall accuracy. That they do not indicates both are limited by the same external factor. The error analysis of Chapter 7 identifies that factor: documents whose gold label is one of several admissible choices. A story about a semiconductor firm's quarterly earnings is legitimately Business and legitimately Sci/Tech; the corpus assigns one label, and no model can recover the other.

This is an irreducible component of the error, a property of the annotation rather than of either model, and it caps what any architecture can achieve on this benchmark.

8.1.4 Cost is where they differ, and the finding inverts expectation

At matched accuracy and matched capacity, the recurrent model trained 1.67 times faster per epoch and performed inference 2.43 times faster.

Section 7.6 analyses why. The summary is that the Transformer's advantage is *asymptotic* — it concerns parallelisability and the growth of sequential depth with sequence length — and that neither condition for realising it holds in this experiment. There is negligible hardware parallelism to exploit on six CPU threads, and at $n = 64$, $d = 128$ the $O(nd^2)$ and $O(n^2d)$ arithmetic costs are of the same order, so self-attention does not even win on operation count.

The honest framing is that this is a result about a *regime*, not about the architectures in the abstract. Two predictions follow, and both are testable: the ordering should reverse on parallel hardware, and the recurrent model's relative cost should grow linearly with sequence length.

8.2 Comparison with published results

Both models fall within the 90–92% band that Table 2.1 identifies for non-pretrained approaches on AG News. The study therefore reproduces the expected performance level, which provides weak corroboration that the pipeline as a whole is sound.

The roughly three-point gap to fine-tuned BERT is instructive and is worth stating plainly, since it is frequently misattributed. That gap is **not** evidence that self-attention is a superior architecture. It is evidence that representations learned from billions of words of unlabelled text are superior to representations learned from 54,000 labelled headlines. The present study isolates architecture by holding pretraining constant at zero for both models, and under that control the architectural difference vanishes entirely.

This distinction matters practically. An engineer choosing between a BiLSTM and a Transformer encoder for a short-text classification task, both trained from scratch, should not expect the accuracy gain reported for BERT to transfer to that choice.

8.3 Implications

For practice. On short-text topic classification at this scale, selecting between these two architectures on expected accuracy is not supported by evidence. The decision should be made on engineering grounds — latency, memory, deployment constraints, tooling — and those costs should be *measured on the target hardware and sequence length* rather than assumed from asymptotic arguments. The cost finding here is a concrete demonstration that the assumed ordering can be wrong.

For evaluation methodology. Reporting a sub-point accuracy difference without a paired significance test is not sound practice. The test applied here cost a few lines of code and reversed the qualitative conclusion, from "the BiLSTM is slightly better" to "the models are indistinguishable".

For from-scratch implementation. The verification suite of Chapter 6 is a transferable methodology. The key insight is that numerical code of this kind fails silently, so verification must consist of equivalence against independent implementations and assertion of mathematical invariants, not merely the observation that training loss decreases.

8.4 Threats to validity

8.4.1 Internal validity

Single seed. This is the most serious limitation. Each architecture was trained once, with one seed. The observed 0.09-point difference is well within typical run-to-run variance for models of this size, which is precisely why Section 7.5 reports a significance test rather than a winner. The conclusion supported is "indistinguishable"; a claim that one architecture is genuinely superior — in either direction — would require multiple seeds and a confidence interval. The study does not make such a claim.

Learning rates differ. Defended in Chapter 4: the controlled quantity is the compute budget, and both rates were selected on validation data. Residual risk remains that a more exhaustive search would favour one architecture.

Both models overfit quickly. The recurrent model peaked after one epoch. Stronger regularisation might alter the comparison, although both models were selected at their respective optima by the early-stopping criterion.

8.4.2 External validity

One corpus, one length regime. All conclusions concern English short news text with a median length of 39 tokens. The central finding is explicitly *conditional* on that length, and the report predicts it would not hold for longer documents.

No pretraining. The findings concern architectures trained from scratch and do not transfer to the pretrained setting.

CPU-only timing. Every figure in Section 7.6 is specific to the hardware of Table 4.5 and would be expected to change on a GPU.

Reduced training set. 60,000 of 120,000 available records were used, per the brief. The more parameter-hungry Transformer might benefit disproportionately from the full corpus.

8.4.3 Construct validity

Truncation. `MAX_LEN` of 64 truncates 3.06% of training documents. This affects both models identically and is unlikely to bias the comparison.

Simple tokenisation. A subword tokeniser would likely raise both models' absolute accuracy. It was omitted to keep the input identical across architectures, at a known cost in absolute performance.

Label noise. As the error analysis shows, a meaningful proportion of test documents admit more than one defensible label. This places a ceiling on measurable accuracy and compresses the range within which architectures can be distinguished — which, it should be acknowledged, makes an equivalence finding easier to obtain. A task with cleaner labels and longer documents would be a more discriminating test.

Chapter 9

Conclusion and Future Work

9.1 Summary

This report has presented a controlled comparison of a bidirectional LSTM and a Transformer encoder on four-class news topic classification, with both architectures implemented from first principles and verified numerically before use.

The experimental design held data, preprocessing, capacity and training budget fixed, so that architecture was the only free variable. Non-embedding parameter counts were matched to within 0.19%. Nine verification checks confirmed the hand-written layers against reference implementations and mathematical invariants, all passing, before any result was generated.

9.2 Findings

Finding 1 — the architectures are statistically indistinguishable in accuracy. 91.14% against 91.05%, with McNemar's exact test returning $p = 0.754$ and the models agreeing on 94.78% of test documents. The observed difference of seven documents out of 7,600 cannot be separated from sampling noise.

Finding 2 — document length explains the equivalence. The median document is 39 tokens. Self-attention's structural advantage is constant-path-length access between distant positions, and a 39-token document contains almost no such distance. The finding is corroborated by the literature: FastText, which models no word order at all, is competitive on this benchmark.

Finding 3 — both models fail identically. Identical per-class difficulty ordering, per-class F_1 agreeing to four decimal places on the hardest class, and a shared dominant confusion between Business and Sci/Tech accounting for roughly a quarter of all errors. Qualitative inspection shows these are documents with more than one defensible label — a property of the annotation, not of either model.

Finding 4 — the cost comparison inverts the expected ordering. The recurrent model trained 1.67 times faster per epoch and performed inference 2.43 times faster. The report attributes this to the absence of the two conditions required for self-attention's asymptotic advantage to materialise: hardware parallelism, and a sequence length exceeding the model dimension.

9.3 Contributions revisited

The study contributes a fully transparent implementation of both architectures; a transferable verification methodology for hand-written numerical layers; a genuinely controlled architectural comparison; a

statistically disciplined negative result; and a cost measurement that contradicts the assumed ordering, together with an analysis of the conditions under which that ordering holds.

9.4 Future work

Multiple seeds. The single most valuable extension. Training both architectures across ten or more seeds would yield confidence intervals and would convert the present "indistinguishable" into a quantified bound on the maximum plausible difference. This is the direct remedy for the principal threat to validity identified in Section 8.4.

Sequence length sweep. The central claim is that the equivalence is conditional on document length. This is directly testable: evaluating both architectures on corpora of systematically increasing document length should show the Transformer's advantage emerging as a function of length. A positive result would confirm the explanatory mechanism; a negative result would refute it.

Parallel hardware. Repeating the cost measurement on a GPU would test the prediction that the observed timing ordering reverses when parallel resources are available.

Ablation studies. Controlled variation of bidirectionality, head count, encoder depth and pooling strategy would separate the contribution of each design decision from the architectural family.

Attention interpretability. Visualising the learned attention distributions would indicate whether the encoder learns locally-concentrated patterns — which would be further evidence for the document-length explanation — or distributes attention broadly.

Pretraining. Comparing a pretrained Transformer against a pretrained recurrent model on identical corpora would isolate the architectural contribution in the pretrained setting, where most practical systems now operate.

9.5 Closing remarks

The most useful result of this study is a negative one, obtained only because a significance test was applied to a difference that looked, on its face, like a result. The second most useful is a cost measurement that contradicted the expectation stated at the outset of the work and was reported as measured rather than adjusted toward the textbook claim.

Both outcomes point the same way. An architecture's advantages are conditional on the task exercising the properties that architecture provides, and on the hardware being able to realise them. Neither condition should be assumed; both can be measured.

Bibliography

[1] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[2] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. ICLR*, 2015.

[3] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[4] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.

[6] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] T. Joachims, "Text categorization with support vector machines," in *Proc. ECML*, 1998.

[9] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proc. EACL*, 2017.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[11] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[13] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proc. EMNLP*, 2015.

[14] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[15] "A study on text classification in the age of large language models," *Machine Learning and Knowledge Extraction*, MDPI, 2024.

[16] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. ICLR Workshop*, 2013.

[17] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. ICML*, 2013.

[18] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.

[19] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. EMNLP*, 2014.

[20] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[21] N. Srivastava et al., "Dropout: A simple way to prevent neural networks from overfitting," *JMLR*, vol. 15, pp. 1929–1958, 2014.

[22] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, 2017.

[23] R. Xiong et al., "On layer normalization in the transformer architecture," in *Proc. ICML*, 2020.

[24] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. NeurIPS*, 2015.

Appendix A

Reproduction

Every number in this report is produced by executing the notebook `ag_news_rnn_vs_transformer.ipynb` end to end. The repository is at `https://github.com/Ashwin-3cS/transformer-t1`.

A.1 Environment

```
uv venv .venv --python 3.14
uv pip install --python .venv/bin/python torch \
    --index-url https://download.pytorch.org/whl/cpu
uv pip install --python .venv/bin/python -r requirements.txt
.venv/bin/python -m ipykernel install --user --name laksh-agnews \
    --display-name "AG News (scratch models)"
```

A.2 Data access

The corpus is downloaded through the Kaggle API on first run and cached in `data/agnews/`. The newer Kaggle token format is used:

```
mkdir -p ~/.kaggle
echo KGAT_xxxxxxxx > ~/.kaggle/access_token
chmod 600 ~/.kaggle/access_token
```

A legacy `/.kaggle/kaggle.json` credential file is also accepted. If neither is present the notebook raises an explanatory error rather than failing with an opaque HTTP status.

A.3 Execution

```
MPLBACKEND=Agg .venv/bin/jupyter nbconvert --to notebook --execute --inplace \
    --ExecutePreprocessor.timeout=7200 ag_news_rnn_vs_transformer.ipynb
```

Total runtime on the reference hardware is approximately nine minutes: 126 s of recurrent model training, 353 s of Transformer training, plus data acquisition, encoding and evaluation.

Setting `SMOKE_TEST = True` in the configuration cell reduces the experiment to 2,000 records and one epoch, allowing the whole notebook to execute in about a minute. This is recommended before committing to a full run.

A.4 Outputs

Execution writes to `artifacts/`: `metrics.json` containing every reported figure in machine-readable form, both model checkpoints, and the four figures reproduced in this report.

A.5 Determinism

All random number generators are seeded at 42. The experiment was executed twice during preparation of this report. Accuracy, macro-F_1, weighted F_1, per-class F_1, parameter counts and the complete McNemar contingency table were identical between runs. Only wall-clock timings varied, as expected.

Appendix B

Complete Classification Reports

Generated on the 7,600-document test set. Support is 1,900 for every class.

B.1 Model A — bidirectional LSTM

	precision	recall	f1-score	support
World	0.9366	0.8789	0.9069	1900
Sports	0.9459	0.9853	0.9652	1900
Business	0.8850	0.8705	0.8777	1900
Sci/Tech	0.8791	0.9111	0.8948	1900
accuracy			0.9114	7600
macro avg	0.9117	0.9114	0.9111	7600
weighted avg	0.9117	0.9114	0.9111	7600

B.2 Model B — Transformer encoder

	precision	recall	f1-score	support
World	0.9227	0.9053	0.9139	1900
Sports	0.9553	0.9674	0.9613	1900
Business	0.8889	0.8674	0.8780	1900
Sci/Tech	0.8754	0.9021	0.8885	1900
accuracy			0.9105	7600
macro avg	0.9106	0.9105	0.9104	7600
weighted avg	0.9106	0.9105	0.9104	7600

B.3 Confusion matrices, absolute counts

Table B.1: Confusion matrix, bidirectional LSTM. Rows are true classes.

	World	**Sports**	**Business**	**Sci/Tech**
World	1670	71	103	56
Sports	9	1872	11	8
Business	58	14	1654	174
Sci/Tech	46	22	101	1731

Table B.2: Confusion matrix, Transformer encoder. Rows are true classes.

	World	**Sports**	**Business**	**Sci/Tech**
World	1720	49	77	54
Sports	27	1838	28	7
Business	55	14	1648	183
Sci/Tech	62	23	101	1714

Appendix C

Source Listings

Complete listings of the from-scratch components. The training harness, preprocessing pipeline and evaluation code are in the notebook and are omitted here for brevity.

C.1 Transformer encoder, complete

```
def sinusoidal_positional_encoding(max_len, d_model):
    position = torch.arange(max_len, dtype=torch.float32).unsqueeze(1)
    even_i   = torch.arange(0, d_model, 2, dtype=torch.float32)
    div_term = torch.exp(-math.log(10000.0) * even_i / d_model)
    pe = torch.zeros(max_len, d_model)
    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)[:, : pe[:, 1::2].shape[1]]
    return pe


class LayerNormScratch(nn.Module):
    def __init__(self, d_model, eps=1e-5):
        super().__init__()
        self.gamma = nn.Parameter(torch.ones(d_model))
        self.beta  = nn.Parameter(torch.zeros(d_model))
        self.eps = eps

    def forward(self, x):
        mu  = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        return self.gamma * (x - mu) / torch.sqrt(var + self.eps) + self.beta


class MultiHeadSelfAttentionScratch(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads, self.d_k = n_heads, d_model // n_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)
```

```
    def _split_heads(self, t):
        B, T, _ = t.shape
        return t.view(B, T, self.n_heads, self.d_k).transpose(1, 2)

    def forward(self, x, key_mask=None, return_attention=False):
        B, T, _ = x.shape
        q = self._split_heads(self.W_q(x))
        k = self._split_heads(self.W_k(x))
        v = self._split_heads(self.W_v(x))

        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.d_k)
        if key_mask is not None:
            scores = scores.masked_fill(key_mask[:, None, None, :] == 0, -1e9)

        attention = torch.softmax(scores, dim=-1)
        context = self.dropout(attention) @ v
        context = context.transpose(1, 2).reshape(B, T, self.n_heads * self.d_k)
        out = self.W_o(context)
        return (out, attention) if return_attention else out


class EncoderBlockScratch(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout):
        super().__init__()
        self.norm1 = LayerNormScratch(d_model)
        self.attention = MultiHeadSelfAttentionScratch(d_model, n_heads, dropout)
        self.norm2 = LayerNormScratch(d_model)
        self.ff_in  = nn.Linear(d_model, d_ff)
        self.ff_out = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, key_mask=None):
        x = x + self.dropout(self.attention(self.norm1(x), key_mask))
        h = self.norm2(x)
        h = self.ff_out(self.dropout(F.gelu(self.ff_in(h))))
        return x + self.dropout(h)


class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, d_model=128, n_heads=4, n_layers=2,
                 d_ff=256, max_len=64, n_classes=4, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=PAD_ID)
        nn.init.normal_(self.embedding.weight, mean=0.0, std=0.1)
        with torch.no_grad():
            self.embedding.weight[PAD_ID].zero_()
        self.register_buffer("pos_encoding",
                             sinusoidal_positional_encoding(max_len, d_model))
        self.dropout = nn.Dropout(dropout)
        self.blocks = nn.ModuleList(
            EncoderBlockScratch(d_model, n_heads, d_ff, dropout)
            for _ in range(n_layers))
        self.norm_final = LayerNormScratch(d_model)
```

```
        self.classifier = nn.Linear(d_model, n_classes)

    def forward(self, ids, lengths=None):
        mask = (ids != PAD_ID)
        T = ids.size(1)
        x = self.embedding(ids) * math.sqrt(self.d_model) \
            + self.pos_encoding[:T].unsqueeze(0)
        x = self.dropout(x)
        for block in self.blocks:
            x = block(x, mask)
        x = self.norm_final(x)
        m = mask.float().unsqueeze(-1)
        pooled = (x * m).sum(dim=1) / m.sum(dim=1).clamp(min=1.0)
        return self.classifier(self.dropout(pooled))
```

C.2 Bidirectional LSTM classifier, complete

```
class BiLSTMClassifier(nn.Module):
    def __init__(self, vocab_size, emb_dim=128, hidden=128,
                 n_classes=4, dropout=0.30):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=PAD_ID)
        nn.init.normal_(self.embedding.weight, mean=0.0, std=0.1)
        with torch.no_grad():
            self.embedding.weight[PAD_ID].zero_()
        self.forward_layer  = LSTMLayerScratch(emb_dim, hidden, reverse=False)
        self.backward_layer = LSTMLayerScratch(emb_dim, hidden, reverse=True)
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Linear(4 * hidden, n_classes)

    def forward(self, ids, lengths=None):
        mask = (ids != PAD_ID).float()
        x = self.embedding(ids)
        out_f, h_f = self.forward_layer(x, mask)
        out_b, h_b = self.backward_layer(x, mask)
        seq = torch.cat([out_f, out_b], dim=-1)
        denom = mask.sum(dim=1, keepdim=True).clamp(min=1.0)
        mean_pooled = (seq * mask.unsqueeze(-1)).sum(dim=1) / denom
        features = torch.cat([mean_pooled, h_f, h_b], dim=-1)
        return self.classifier(self.dropout(features))
```

Appendix D

Verification Suite Output

Output produced on every execution, before training begins.

```
check                                                result  detail
-------------------------------------------------------------------------------
LSTM cell matches nn.LSTM                            PASS    outputs=5.96e-08
                                                             final h=4.47e-08
LayerNorm matches F.layer_norm                       PASS    max|diff|=4.77e-07
Attention matches naive per-head reference           PASS    max|diff|=8.94e-08
Attention rows sum to 1                              PASS    max|rowsum-1|=1.79e-07
Padded keys get ~zero attention                      PASS    max mass=0.00e+00
BiLSTM: logits unchanged by extra padding            PASS    max|diff|=0.00e+00
Transformer: logits unchanged by extra padding       PASS    max|diff|=1.19e-07
BiLSTM: overfits 200 examples                        PASS    accuracy=100.0%
Transformer: overfits 200 examples                   PASS    accuracy=100.0%
-------------------------------------------------------------------------------
All 9 checks passed. The hand-written layers are numerically correct.
```

Appendix E

Summary of Key Figures

Table E.1: All headline quantities reported in this study.

Quantity	**BiLSTM**	**Transformer**
Test accuracy	0.9114	0.9105
Macro F_1	0.9111	0.9104
Weighted F_1	0.9111	0.9104
Test cross-entropy	0.2797	0.2944
Best validation macro-F_1	0.9091	0.9077
Best epoch	1	3
Epochs run	3	5
Seconds per epoch	41.2	68.6
Total training time (s)	126	353
Inference (ms / 1000 docs)	138	335
Total parameters	4,612,612	4,613,124
Non-embedding parameters	265,220	265,732

Table E.2: Shared experimental quantities.

Quantity	**Value**
Training records	54,000
Validation records	6,000
Test records	7,600
Vocabulary size	33,964
Maximum sequence length	64
Median document length	39 tokens
Coverage at max length	96.94%
Verification checks passed	9 / 9
Model agreement	94.78%
Discordant documents	367
McNemar p-value	0.754